Compact CHI 架构说明文档

适用于 KunminghuV3（内部版本）

d0f2609
2026 年 8 月 28 日

香山开源处理器社区
XiangShan Open-Source Processor Community

前言

本文档为 Compact CHI 总线协议架构说明文档（草稿）。内部版本，切勿外传。

声明

版权所有 © 2024 - 2025 香山开源处理器团队·北京开源芯片研究院

本文档采用 “知识共享署名 4.0” 协议公开发布。您可以根据该协议的规定，自由地使用、修改、分发本文档，但必须给出恰当的署名、表明原始许可协议、标注是否对原始文档作了修改。具体许可条款以知识共享组织公布的完整法律文本为准。

本文档仅提供阶段性信息，可能不定期根据实际情况更新。除非另有约定，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

版本信息

日期	版本	说明
2025-04-30	Issue A	内部版本

目录

第一部分　介绍 **1**

1　架构概述 **2**
1.1　组件 2
1.2　架构层级 2

2　拓扑结构 **4**

3　参数 **5**

4　通道 **7**
4.1　通道间依赖规则 8

5　事务分类 **9**
5.1　非驱逐请求 9
5.1.1　非 Cacheable 读 9
5.1.2　Cacheable 读 9
5.1.3　无数据权限转移 9
5.1.4　缓存行维护（CMO） 9
5.1.5　跨层预取 9
5.1.6　非 Cacheable 写 9
5.1.7　Cacheable 写 10
5.1.8　原子操作 10
5.1.9　Snoop 10
5.2　驱逐请求 10
5.2.1　缓存行写回 10
5.2.2　缓存行踢出 10

6　组件分类 **11**
6.1　上游 11
6.1.1　Type 1 (Fully coherent) 11
6.1.2　Type 2 (Read-only coherent) 11
6.1.3　Type 3 (Non-coherent) 12
6.1.4　Type 4 (Read-only Non-coherent) 13
6.1.5　Type 5 (Lower-level Prefetcher) 14
6.2　下游 14

第二部分 一致性协议 15

7 一致性状态 16
7.1 一致性模型 . . . 16
7.2 缓存行状态 . . . 16

8 请求类型 18
8.1 非 Cacheable 读 . . . 18
8.2 Cacheable 读 . . . 18
8.3 缓存行踢出 . . . 19
8.4 缓存行写回 . . . 19
8.5 无数据权限转移 . . . 20
8.6 缓存行维护（CMO） . . . 20
8.7 跨层预取 . . . 21
8.8 非 Cacheable 写 . . . 22
8.9 Cacheable 写 . . . 22
8.10 原子操作 . . . 23
8.11 Snoop . . . 23

第三部分 事务 25

9 通道定义 26
9.1 Flit 定义 . . . 26
9.1.1 EVT 通道 . . . 26
9.1.2 REQ 通道 . . . 26
9.1.3 SNP 通道 . . . 27
9.1.4 RSP 通道 . . . 27
9.1.5 DAT 通道 . . . 28
9.2 域定义 . . . 29
9.2.1 事务标识, TxnID . . . 29
9.2.2 来源节点标识，SrcID . . . 29
9.2.3 目标节点标识，TgtID . . . 29
9.2.4 返回事务标识，DBID . . . 29
9.2.5 请求/消息类型, Opcode . . . 29
9.2.6 事务数据长度, Size . . . 32
9.2.7 事务地址，Addr . . . 32
9.2.8 别名地址，TagAlias . . . 32
9.2.9 地址隔离保留，NS . . . 32
9.2.10 定序属性，Order . . . 32
9.2.11 内存属性，MemAttr . . . 32
9.2.12 独占访问标识，Excl . . . 32
9.2.13 期望 CompData 回复，ExpCompData . . . 32
9.2.14 期望 CompStash 回复，ExpCompStash . . . 32

9.2.15 路索引有效，WayValid . . . 32
9.2.16 路索引，Way . . . 32
9.2.17 状态回复, Resp . . . 32
9.2.18 错误信息，RespErr . . . 33
9.2.19 数据来源标识，DataSource . . . 33
9.2.20 下游忙碌信息，CBusy . . . 33
9.2.21 循迹标识，TraceTag . . . 33

10 事务结构 34
10.1 非 Cacheable 读 . . . 34
10.2 Cacheable 读 . . . 34
10.3 缓存行踢出 . . . 36
10.4 缓存行写回 . . . 37
10.5 无数据权限转移 . . . 38
10.6 缓存行维护（CMO） . . . 38
10.7 跨层预取 . . . 39
10.8 非 Cacheable 写 . . . 40
10.9 Cacheable 写 . . . 41
10.10 原子操作 . . . 42
10.11 Snoop . . . 42

11 上游路持久化 45

12 上游路预测 47

第四部分 链路层 48

13 时钟与初始状态 49
13.1 时钟 . . . 49
13.2 复位 . . . 49
13.3 初始化 . . . 49

14 协议资源流控 50
14.1 P-Credit 流控 . . . 50
14.1.1 动态 P-Credit 及其同步 . . . 51
14.1.2 静态 P-Credit . . . 53
14.2 基于资源的请求 Retry . . . 53
14.3 基于冲突的请求 Retry . . . 56
14.4 参考设计 . . . 56
14.4.1 通道 . . . 56
14.4.2 端口 . . . 57
14.4.3 P-Credit 资源绑定 . . . 59
14.4.4 基于地址冲突的请求 Retry . . . 60

15 链路 QoS 61

15.1 与 P-Credit 耦合 62
15.2 参考设计 63
15.2.1 Requester 侧等待率相关的链路 QoS 63

图目录

6.1 Type 1 接口 11
6.2 Type 2 接口 12
6.3 Type 3 接口 12
6.4 Type 4 接口 13
6.5 Type 5 接口 14

7.1 缓存行状态属性与一致性状态的映射关系 16

10.1 非 Cacheable 读事务流程 34
10.2 Cacheable 读事务流程 35
10.3 缓存行踢出事务流程 37
10.4 缓存行写回事务流程 37
10.5 无数据权限转移 38
10.6 缓存行维护事务流程 39
10.7 跨层预取事务流程 40
10.8 非 Cacheable 写事务流程 40
10.9 Cacheable 写事务流程 41
10.10 Snoop 事务流程 42

11.1 上游路持久化实现例 46

14.1 动态 P-Credit 的传递与同步信号 51
14.2 动态 P-Credit 收回时可能的竞争情况时序示例图 52
14.3 静态 P-Credit 的传递信号 53
14.4 P-Credit Retry 机制信号 54
14.5 P-Credit Retry 消息信号时序示例图 55
14.6 动态流控通道参考设计 56
14.7 静态流控通道参考设计 57
14.8 无阻塞通道参考设计 57
14.9 上游端口参考设计 58
14.10 Completer 侧 P-Credit 资源绑定参考设计 59

15.1 链路 QoS 的传递信号 61
15.2 链路 QoS 信号时序示例图 62
15.3 Requester 侧等待率相关链路 QoS 参考设计 63

表目录

4.1 各通道及其在上游、下游通路上的映射 7

8.1 非 Cacheable 读请求发送时允许的 Requester 缓存一致性状态 18
8.2 非 Cacheable 读请求结束时允许的 Requester 最终缓存一致性状态 18
8.3 Cacheable 读请求发送时允许的 Requester 缓存一致性状态 19
8.4 Cacheable 读请求结束时允许的 Requester 最终缓存一致性状态 19
8.5 缓存行踢出请求发送时允许的 Requester 缓存一致性状态 19
8.6 缓存行踢出请求结束时允许的 Requester 最终缓存一致性状态 19
8.7 缓存行写回请求发送时允许的 Requester 缓存一致性状态 20
8.8 缓存行写回请求结束时允许的 Requester 最终缓存一致性状态 20
8.9 无数据权限转移请求发送时允许的 Requester 缓存一致性状态 20
8.10 无数据权限转移请求结束时允许的 Requester 最终缓存一致性状态 20
8.11 缓存行维护请求发送时允许的 Requester 缓存一致性状态 21
8.12 缓存行维护请求结束时允许的 Requester 最终缓存一致性状态 21
8.13 跨层预取请求发送时允许的 Requester 缓存一致性状态 21
8.14 跨层预取请求结束时允许的 Requester 最终缓存一致性状态 22
8.15 非 Cacheable 写请求发送时允许的 Requester 缓存一致性状态 22
8.16 非 Cacheable 写请求结束时允许的 Requester 最终缓存一致性状态 22
8.17 Cacheable 写请求发送时允许的 Requester 缓存一致性状态 23
8.18 Cacheable 写请求结束时允许的 Requester 最终缓存一致性状态 23

14.1 动态 P-Credit 的传递与同步信号列表 51
14.2 静态 P-Credit 53
14.3 P-Credit Retry 机制信号列表 54
14.4 Completer 侧 P-Credit 资源映射表参考设计 59
14.5 请求与 P-Credit 资源映射表参考设计 60

15.1 链路 QoS 信号列表 61
15.2 链路 QoS 信号值组合 61
15.3 与 P-Credit 耦合的链路 QoS 信号列表示例 62

第一部分

介绍

1 架构概述

Compact CHI 是由 CHI 架构演化而来的一个紧凑型变体，且不完全是 CHI 的子集。Compact CHI 的目标是提供一种适用于在小型紧凑系统中连接多个一致性节点的总线标准，并主要特化为提供 Core、L1 Cache 与 L2 Cache 之间的连接。

1.1 组件

基于 Compact CHI 的系统主要由处理器核、L1 Cache、L2 Cache、集群/簇控制器以及互联本身组成，并允许包含带外信号以支持系统控制功能。

该架构有以下主要特性：

- 层次化实现，解耦协议、传输层
- 基于封包与 Credit 的网络流通信
- 一致性协议支持：
  - 64 Bytes 为粒度的缓存行
  - Snoop Filter 与 Directory
  - 支持 MESI 协议
- 一致性事务支持：
  - 原子性操作
  - 独占访问（LR/SC）优化
- 通过请求重试进行的动态协议层资源管理

1.2 架构层级

本架构分为以下层级：

- 协议层

- 传输层

下表说明了各个层级的主要功能：

层级	粒度	主要功能
协议层	事务	· 产生并处理节点间的请求与回复 · 定义各个一致性节点上的合法缓存状态变化 · 定义协议流程 · 管理协议层的流控（重试等）
链路层	封包	· 以封包的形式传输协议消息 · 在多个节点间共享资源利用情况 · 实现流控

2　拓扑结构

3 参数

- **TxnID_Width**
  来自上游的 TxnID 的数据宽度，由上游所有单个节点的最大事务并行度决定。
- **DBID_Width**
  来自下游的 TxnID/DBID 的数据宽度，由下游所有单个节点的最大事务并行度决定。
- **WayIndex_Width**
  路索引（Way）的数据宽度，由下游所有单个节点的最大缓存路数决定。
  - 未定义或设定为 0 值则表示该系统中不支持 Way 相关的所有特性
  - 设定为非 0 值则表示该系统支持或不支持 Way 的相关特性，若总是不支持则 WayValid 总是无效
    参见 **UWPersist_Enable** 与 **UWPredit_Enable**。
    参见 **上游路持久化**章节与 **上游路预测**章节。
- **UpstreamNodeID_Width**
  上游 Node ID 的数据宽度，由上游的节点数量与路由方式决定。
- **DownstreamNodeID_Width**
  下游 Node ID 的数据宽度，由下游的节点数量与路由方式决定。
- **Data_Width**
  单次传输的缓存行数据宽度，即总线数据宽度。
  - 允许的宽度为 64, 128, 256, 512。
  - 默认宽度为 256。
- **TagAlias_Width**
  缓存行别名地址的宽度，主要用于 VIPT 上游缓存系统的虚拟地址别名处理，通常由一致性 L1 D-Cache、L1 I-Cache 的具体实现决定。
  - 设定为 0 值表示该系统不需要支持硬件维护的上游虚拟地址别名处理
  - 设定为非 0 值表示该系统中的下游必须支持上游虚拟地址别名处理，但上游可以将其设定为任意固定值而不使用此功能
- **UWPersist_Enable**
  是否启用上游路持久化。

  - 设定为 false 则意味着关闭上游路持久化，对应的 WayValid 总是无效

  - 设定为 true 则意味着下游可以接受上游路持久化的 Way 信息，若上游选择不持久化则对应的 WayValid 总是无效，且下游总是可以在 WayValid 有效时自行决定是否使用该 Way
    参见 **上游路持久化**章节。

- **UWPredict_Enable**
  是否启用上游路预测。

  - 设定为 false 则意味着关闭上游路预测，对应的 WayValid 总是无效

  - 设定为 true 则意味着下游可以接受上游路预测的 Way 信息，若上游选择不进行路预测则对应的 WayValid 总是无效，且下游总是可以在 WayValid 有效时自行决定是否使用该 Way；在上游使用该 Way 时，必须假设该信息可能是错的，并实现相关的回滚或重发操作
    参见 **上游路预测**章节。

4 通道

在本规范中，链路层定义并提供了多组通道用于事务消息的通信。每个通道都有自己单独的域格式定义。

表 4.1: 各通道及其在上游、下游通路上的映射

通道	说明	用途	上游通道	下游通道
EVT Eviction	发起驱逐的起始消息	所有的上游驱逐请求	TXEVT	RXEVT
REQ Request	发起请求的起始消息	所有的上游非驱逐请求	TXREQ	RXREQ
RSP Response	不携带数据的回复消息	来自下游的非数据回复 来自上游的 Snoop 回复与 CompAck	RXRSP TXRSP	TXRSP RXRSP
SNP Snoop	Snoop 请求的起始消息	所有的下游 Snoop 请求	RXSNP	TXSNP
DAT Data	携带数据的回复消息	来自上游的写数据与 Snoop 数据回复 来自下游的读数据回复	TXDAT RXDAT	RXDAT TXDAT

对于 EVT 通道：

- 上游组件在 EVT 通道为 Requester，通道名称为 TXEVT
- 下游组件在 EVT 通道为 Completer，通道名称为 RXEVT

对于 REQ 通道：

- 上游组件在 REQ 通道为 Requester，通道名称为 TXREQ
- 下游组件在 REQ 通道为 Completer，通道名称为 RXREQ

对于 SNP 通道：

- 上游组件在 SNP 通道为 Completer，通道名称为 RXSNP
- 下游组件在 SNP 通道为 Requester，通道名称为 TXSNP

4.1 通道间依赖规则

对于 Requester，通道之间的依赖必须遵守以下规则：

- 来自 RXSNP 的进度不可以依赖于 TXREQ 的进度
- 来自 RXSNP 的进度可以依赖于 TXRSP 的进度
- 来自 RXSNP 的进度可以依赖于 TXDAT 的进度
- 来自 RXRSP 的进度不可以依赖于其他任何通道的进度
- 来自 RXDAT 的进度不可以依赖于其他任何通道的进度

对于 Completer，通道之间的依赖必须遵守以下规则：

- 来自 RXEVT 的进度不可以依赖于 TXSNP 的进度
- 来自 RXREQ 的进度可以依赖于 TXSNP 的进度
- 来自 RXEVT 的进度可以依赖于 TXRSP 的进度
- 来自 RXEVT 的进度可以依赖于 TXDAT 的进度
- 来自 RXREQ 的进度可以依赖于 TXRSP 的进度
- 来自 RXREQ 的进度可以依赖于 TXDAT 的进度
- 来自 RXRSP 的进度不可以依赖于其他任何通道的进度
- 来自 RXDAT 的进度不可以依赖于其他任何通道的进度

5 事务分类

Compact CHI 定义了各种事务，其主要分类如下：

5.1 非驱逐请求

此类请求的首个消息使用 REQ 通道。

5.1.1 非 Cacheable 读

- ReadNoSnp

5.1.2 Cacheable 读

- ReadOnce
- ReadShared
- ReadUnique

5.1.3 无数据权限转移

- MakeUnique

5.1.4 缓存行维护（CMO）

- CleanShared
- CleanInvalid
- MakeInvalid

5.1.5 跨层预取

- StashUnique
- StashShared

5.1.6 非 Cacheable 写

- WriteNoSnpPtl
- WriteNoSnpFull

5.1.7 Cacheable 写

- WriteUniqueFull

5.1.8 原子操作

- AtomicStore
- AtomicLoad
- AtomicSwap
- AtomicCompare

5.1.9 Snoop

- SnpMakeInvalid
- SnpToInvalid
- SnpToShared
- SnpToClean

5.2 驱逐请求

此类请求的首个消息使用 EVT 通道。

5.2.1 缓存行写回

- WriteBackFull

5.2.2 缓存行踢出

- Evict

6 组件分类

在本规范中，组件分为上游组件和下游组件。每一个组件既可以是 Requester 又可以是 Completer。请求的最先消息发起方称为 Requester，接收请求方称为 Completer。

6.1 上游

每一个上游组件只可以连接一个下游组件。上游组件一般为 L1 Cache、PTW 等。根据其可能会需要的不同操作、不同缓存一致性要求和等级，将上游组件分为四个类型。

6.1.1 Type 1 (Fully coherent)

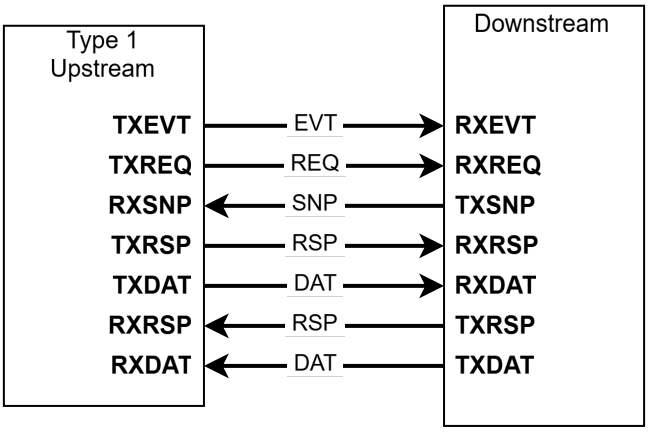


图 6.1: Type 1 接口

对于 Type 1 组件的接口定义如图所示。Type 1 组件又称 Fully coherent 组件，支持所有类型的事务。Type 1 组件需要维护完整的自身一致性状态。

仅 Type 1 组件支持 CMO 事务。

通常来说，L1 D-Cache 为 Type 1 组件。

6.1.2 Type 2 (Read-only coherent)

对于 Type 2 组件的接口定义如图所示。Type 2 组件又称 Read-only coherent 组件，仅支持以下类型的事务：

- 非 Cacheable 读
  - ReadNoSnp

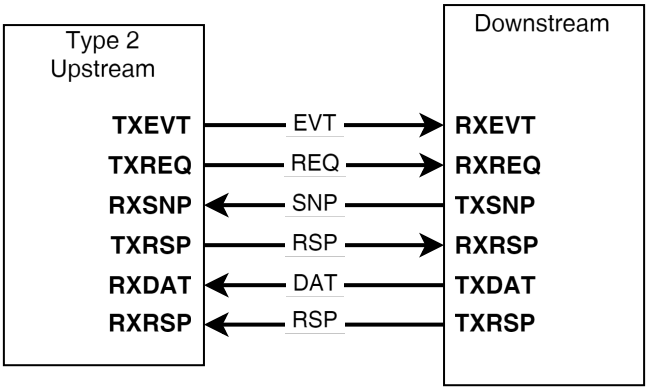


图 6.2: Type 2 接口

- 部分 Cacheable 读
  - ReadOnce
  - ReadShared（仅限得到任意的 Shared 状态，不允许得到 Unique 状态）
- 替换
  - Evict
- 跨层预取
  - StashShared
  - StashUnique

Type 2 组件不允许得到写权限，即 Completer 不可以为 Type 2 Requester 提供 Unique 状态。Type 2 组件仅需要维护 Shared 的自身一致性状态。

Type 2 组件需要支持的 Snoop 事务类型：

- SnpMakeInvalid

通常来说，支持缓存一致性的 L1 I-Cache、一致性的 TLB 为 Type 2 组件。

6.1.3 Type 3 (Non-coherent)

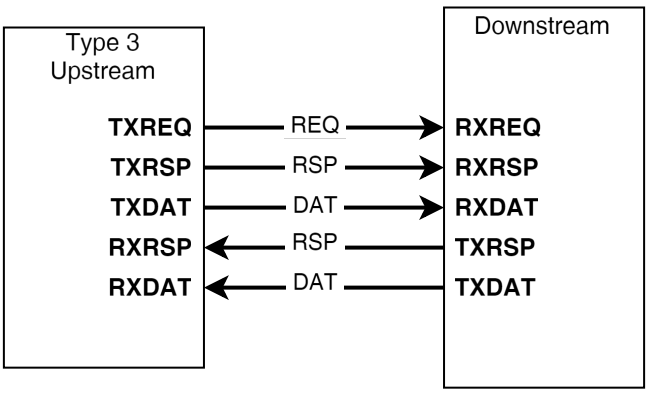


图 6.3: Type 3 接口

对于 Type 3 组件的接口定义如图所示。Type 3 组件又称 Non-coherent 组件，仅支持以下类型的事务：

- 非 Cacheable 读
  - ReadNoSnp
- 部分 Cacheable 读
  - ReadOnce
- 非 Cacheable 写
  - WriteNoSnpPtl
  - WriteNoSnpFull
- 部分 Cacheable 写
  - WriteUniqueFull
- 跨层预取
  - StashShared
  - StashUnique

Type 3 组件不需要维护自身的一致性状态。
通常来说，独立的 MMIO 控制器、内嵌启动处理器、独立的预取器为 Type 3 组件。

6.1.4 Type 4 (Read-only Non-coherent)

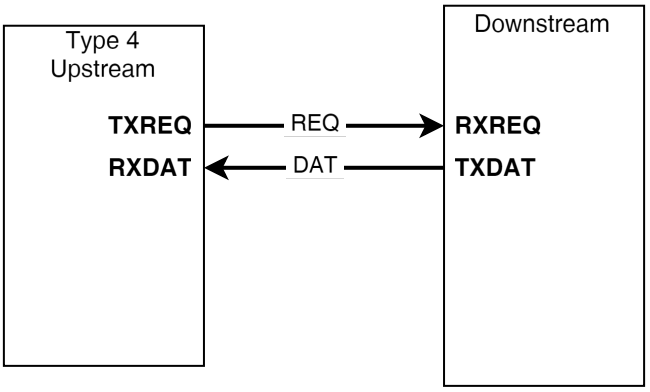


图 6.4: Type 4 接口

对于 Type 4 组件的接口定义如图所示。Type 4 组件又称 Read-only Non-coherent 组件，仅支持以下类型的事务:

- 非 Cacheable 读
  - ReadNoSnp
- 部分 Cacheable 读
  - ReadOnce
- 跨层预取
  - StashShared

- StashUnique

通常来说，非一致性 PTW、独立的预取器为 Type 4 组件。

6.1.5 Type 5 (Lower-level Prefetcher)

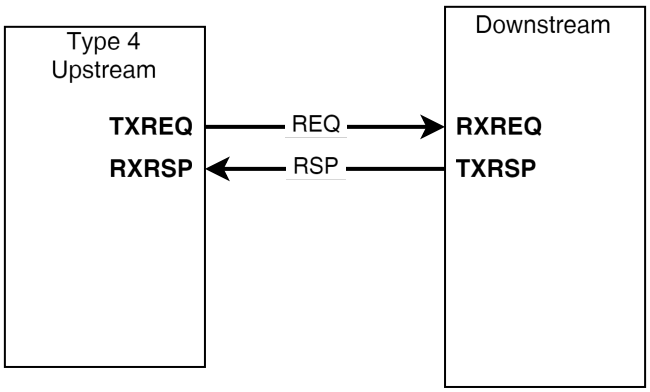


图 6.5: Type 5 接口

对于 Type 5 组件的接口定义如图所示。Type 5 组件又称 Lower-level Prefetcher 组件，仅支持以下类型的事务：

· 跨层预取

- StashShared
- StashUnique

通常来说，独立的预取器为 Type 5 组件。

6.2 下游

每一个下游组件可以连接多个上游组件，其通道的定义根据其连接的上游组件类型而定。下游组件一般为 L2 Cache。

第二部分

一致性协议

7 一致性状态

7.1 一致性模型

基于缓存行的一致性需求，每一种一致性状态包含了以下的属性：

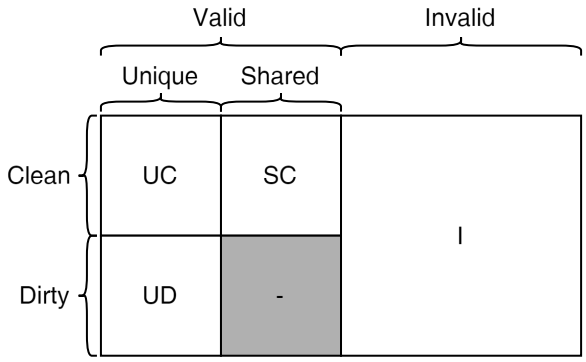


图 7.1: 缓存行状态属性与一致性状态的映射关系

- **Valid, Invalid**
  - 若为 Valid，表示缓存行存在于当前缓存组件中
  - 若为 Invalid，表示缓存行不存在于当前缓存组件中
- **Unique, Shared**
  - 若为 Unique，表示缓存行的有效数据仅可能存在于当前缓存组件中，此时拥有读、写缓存行数据的能力
  - 若为 Shared，表示缓存行的有效数据可能存在于多个缓存组件中，此时拥有读缓存行数据的能力
- **Clean, Dirty**
  - 若为 Clean，表示当前缓存行不需要最终负责写回缓存行数据，持有的数据可能与内存中的不同
  - 若为 Dirty，表示当前缓存行需要最终负责写回缓存行数据，持有的数据可能与内存中的不同

7.2 缓存行状态

任意一个缓存组件实现的缓存行状态必须是以下所有状态的集合或其子集：

- **I** Invalid：
  - 缓存行不存在于当前缓存组件中

- **UC** Unique Clean：
  - 缓存行当前存在于当前缓存组件中
  - 缓存行的有效数据仅存在于当前缓存组件中
  - 缓存行的有效数据可以在当前缓存组件中被直接修改（即有写数据能力）
- **UD** Unique Dirty：
  - 缓存行当前存在于当前缓存组件中
  - 缓存行的有效数据仅存在于当前缓存组件中
  - 缓存行的有效数据可以在当前缓存组件中被直接修改（即有写数据能力）
  - 缓存行的有效数据已经被修改，且最终必须在替换、踢出时将数据写回（除非被直接 Invalidated）
- **SC** Shared Clean：
  - 缓存行当前存在于当前缓存组件中
  - 缓存行的有效数据可能存在于多个缓存组件中
  - 缓存行的有效数据不可以在当前缓存组件中被直接修改（即没有写数据能力）
  - 缓存行的有效数据不需要被写回

8 请求类型

8.1 非 Cacheable 读

- **ReadNoSnp**
  向一个非 Cacheable 地址发起的读请求。若向 Cacheable 地址发起了此请求，数据可能不来自任何的缓存行或不反映任何时刻的最新数据。

对于非 Cacheable 读请求，Requester 在发送请求时刻允许的状态如下表所示：

表 8.1: 非 Cacheable 读请求发送时允许的 Requester 缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
ReadNoSnp	Y	-	-	-

对于非 Cacheable 读请求，Requester 在事务结束时刻允许的最终状态如下表所示：

表 8.2: 非 Cacheable 读请求结束时允许的 Requester 最终缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
ReadNoSnp	Y	-	-	-

8.2 Cacheable 读

- **ReadOnce**
  向一个 Cacheable 地址发起的读请求，用以获得目标地址在缓存系统中某一时刻最新数据的快照。请求的发起者在得到数据后不保留任何的一致性状态。

- **ReadShared**
  向一个 Cacheable 地址发起的读请求，用以对目标地址进行 Load 动作。
  Requester 可以接受以下的所有状态：

  – UC, UD, SC

- **ReadUnique**
  向一个 Cacheable 地址发起的读请求，用以对目标地址进行 Store 动作。若 Requester 本身持有 Shared 状态，且在 ReadUnique 期间没有丢失原先的 Shared 状态与数据（收到 Invalidating Snoop），那么 ReadUnique 结束时允许仅提供新的一致性状态，而不再提供数据。
  Requester 可以接受以下的所有状态：

– UC, UD

对于 Cacheable 读请求，Requester 在发送请求时刻允许的状态如下表所示：

表 8.3: Cacheable 读请求发送时允许的 Requester 缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
ReadOnce	Y	-	-	-
ReadShared	Y	Y	-	-
ReadUnique	Y	Y	Y	Y

对于 Cacheable 读请求，Requester 在事务结束时刻允许的最终状态如下表所示：

表 8.4: Cacheable 读请求结束时允许的 Requester 最终缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
ReadOnce	Y	-	-	-
ReadShared	-	Y	Y	Y
ReadUnique	-	-	Y	Y

8.3 缓存行踢出

· **Evict**
通知目标地址的 Clean 缓存行被踢出。

对于缓存行踢出请求，Requester 在发送请求时刻允许的状态如下表所示：

表 8.5: 缓存行踢出请求发送时允许的 Requester 缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
Evict	Y	-	-	-

对于缓存行踢出请求，Requester 在事务结束时刻允许的最终状态如下表所示：

表 8.6: 缓存行踢出请求结束时允许的 Requester 最终缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
Evict	Y	-	-	-

8.4 缓存行写回

· **WriteBackFull**
写回一整个缓存行的 Dirty 状态数据。

- 若写回时 CopyBackWrData 携带的状态不为 Invalid，数据中的每一个字节都必须是有效的，即 BE 位必须覆盖整个缓存行
- 可作为 Completer 不需要进行数据合并的提示

对于缓存行写回请求，Requester 在发送请求时刻允许的状态如下表所示：

表 8.7: 缓存行写回请求发送时允许的 Requester 缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
WriteBackFull	-	-	-	Y

对于缓存行写回请求，Requester 在事务结束时刻允许的最终状态如下表所示：

表 8.8: 缓存行写回请求结束时允许的 Requester 最终缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
WriteBackFull	Y	-	-	-

8.5 无数据权限转移

· **CleanUnique**
目前暂时仅作保留符号，与 MakeUnique 等价。

· **MakeUnique**
向一个 Cacheable 地址发起的请求，用以对目标地址进行 Store 动作，且不读取该目标地址的缓存行数据。此请求只允许在确定要对整个缓存行进行全覆盖写时被使用。缓存系统中其它所有同目标地址的数据都会被直接无效化，且不会产生任何数据写回。

对于无数据权限转移请求，Requester 在发送请求时刻允许的状态如下表所示：

表 8.9: 无数据权限转移请求发送时允许的 Requester 缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
MakeUnique	Y	Y	Y	Y

对于无数据权限转移请求，Requester 在事务结束时刻允许的最终状态如下表所示：

表 8.10: 无数据权限转移请求结束时允许的 Requester 最终缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
MakeUnique	-	-	-	Y

8.6 缓存行维护（CMO）

在所有的缓存行维护事务中，收到其 CompCMO 回复时，表示所有脏数据最终都会在一段时间内被写回内存或任意 Non-coherent 组件，且该时刻已经保证 CleanShared 的结果对所有 I/O coherent 组件可见 [**注**]。

- **CleanShared**
  使一个 Cacheable 地址的所有缓存行进入 Clean 状态，任何包含 Dirty 状态的目标地址缓存行数据都会被写回。

- **CleanInvalid**
  使一个 Cacheable 地址的所有缓存行进入 Invalid 状态，任何包含 Dirty 状态的目标地址缓存行数据都会被写回。

- **MakeInvalid**
  使一个 Cacheable 地址的所有缓存行进入 Invalid 状态，任何目标地址缓存行数据都会被直接丢弃。

对于缓存行维护请求，Requester 在发送请求时刻允许的状态如下表所示：

表 8.11: 缓存行维护请求发送时允许的 Requester 缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
CleanShared	Y	Y	Y	Y
CleanInvalid	Y	Y	Y	Y
MakeInvalid	Y	Y	Y	Y

对于缓存行维护请求，Requester 在事务结束时刻允许的最终状态如下表所示：

表 8.12: 缓存行维护请求结束时允许的 Requester 最终缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
CleanShared	Y	Y	Y	-
CleanInvalid	Y	-	-	-
MakeInvalid	Y	-	-	-

8.7 跨层预取

- **StashUnique**
  对一个 Cacheable 地址的缓存行进行跨层预取，且预取的意图为将要进行 Store 动作，其预取的目标为非自身的下层缓存结构（例如 L2、L3 等）。

- **StashShared**
  对一个 Cacheable 地址的缓存行进行跨层预取，且预取的意图为将要进行 Load 动作，其预取的目标为非自身的下层缓存结构（例如 L2、L3 等）。

对于跨层预取请求，Requester 在发送请求时刻允许的状态如下表所示：

表 8.13: 跨层预取请求发送时允许的 Requester 缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
StashUnique	Y	Y	Y	Y

请求类型	I	SC	UC	UD
StashShared	Y	Y	Y	Y

对于跨层预取请求，Requester 在事务结束时刻允许的最终状态如下表所示：

表 8.14: 跨层预取请求结束时允许的 Requester 最终缓存一致性状态

请求类型	I	SC	UC	UD
StashUnique	Y	Y	Y	Y
StashShared	Y	Y	Y	Y

8.8 非 Cacheable 写

- **WriteNoSnpFull**
  对一个非 Cacheable 地址写入宽度为一整个缓存行的数据：
  - 数据中的每一个字节都必须是有效的，即 BE 位必须覆盖整个缓存行
- **WriteNoSnpPtl**
  对一个非 Cacheable 地址写入宽度最大为一整个缓存行的数据：
  - 数据中的部分字节是有效的（包括所有字节有效和所有字节无效的情况），即 BE 必须覆盖有效的数据，但允许不总是整个缓存行

对于非 Cacheable 写请求，Requester 在发送请求时刻允许的状态如下表所示：

表 8.15: 非 Cacheable 写请求发送时允许的 Requester 缓存一致性状态

请求类型	I	SC	UC	UD
WriteNoSnpFull	Y	-	-	-
WriteNoSnpPtl	Y	-	-	-

对于非 Cacheable 写请求，Requester 在事务结束时刻允许的最终状态如下表所示：

表 8.16: 非 Cacheable 写请求结束时允许的 Requester 最终缓存一致性状态

请求类型	I	SC	UC	UD
WriteNoSnpFull	Y	-	-	-
WriteNoSnpPtl	Y	-	-	-

8.9 Cacheable 写

- **WriteUniqueFull**
  对一个 Cacheable 地址写入宽度为一整个缓存行的数据，且自身该目标地址的缓存行处于 Invalid 状态：

  - 数据中的每一个字节都必须是有效的，即 BE 位必须覆盖整个缓存行

- **WriteUniquePtl**
  对一个 Cacheable 地址写入宽度最大为一整个缓存行的数据，且自身该目标地址的缓存行处于 Invalid 状态：

  - 数据中的部分字节是有效的（包括所有字节有效和所有字节无效的情况），即 BE 必须覆盖有效的数据，但允许不总是整个缓存行

对于 Cacheable 写请求，Requester 在发送请求时刻允许的状态如下表所示：

表 8.17: Cacheable 写请求发送时允许的 Requester 缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
WriteUniqueFull	Y	-	-	-
WriteUniquePtl	Y	-	-	-

对于 Cacheable 写请求，Requester 在事务结束时刻允许的最终状态如下表所示：

表 8.18: Cacheable 写请求结束时允许的 Requester 最终缓存一致性状态

请求类型	**I**	**SC**	**UC**	**UD**
WriteUniqueFull	Y	-	-	-
WriteUniquePtl	Y	-	-	-

8.10 原子操作

- **AtomicStore**
- **AtomicLoad**
- **AtomicSwap**
- **AtomicCompare**

8.11 Snoop

- **SnpMakeInvalid**
  请求目标地址的缓存行状态变为至少 Invalid，并直接丢弃所有 Dirty 状态的数据：

  - 不允许向 Requester 返回数据
  - 必须将 Completer 内目标缓存行的一致性状态转换到 Invalid 状态

- **SnpToInvalid**
  请求目标地址的缓存行状态变为至少 Invalid，并且拿回 Dirty 状态的数据：

  - 在 Completer 内缓存行状态包含 Dirty 状态时，必须向 Requester 返回数据
  - 必须将 Completer 内目标缓存行的一致性状态转换到 Invalid 状态

- **SnpToShared**
  请求目标地址的缓存行状态变为至少 Shared，并且拿回 Dirty 状态的数据：
  - 在 Completer 内目标缓存行状态包含 Dirty 状态时，必须向 Requester 返回数据
  - 在 Completer 内目标缓存行的最终状态不允许包含 Unique 状态
- **SnpToClean**
  请求目标地址的缓存行状态变为至少 Unique Clean，并且拿回 Dirty 状态的数据：
  - 在 Completer 内目标缓存行状态包含 Dirty 状态时，必须向 Requester 返回数据
  - 在 Completer 内目标缓存行的最终状态不允许包含 Dirty 状态

第三部分

事务

9 通道定义

9.1 Flit 定义

9.1.1 EVT 通道

域名称	宽度	说明
TxnID	`TxnID_Width`	每组端口可以有不同的 TxnID 宽度
SrcID	`UpstreamNodeID_Width`	-
TgtID	`DownstreamNodeID_Width`	-
Opcode	0 or 1	对于 Type 2 组件，总是 Evict；总宽度为 1，因不足位数导致的端口转换必须采用零扩展
Addr	48	-
NS	1	-
MemAttr	1	仅含 Allocate 属性
WayValid	0 or 1, **由 `UWPersist_Enable` 决定**	**路持久化值**是否有效，仅在上下游同时支持**路持久化**时存在
Way	0 or `WayIndex_Width`	根据下游缓存结构决定，仅在上下游同时支持**路持久化**时存在
TraceTag	1	-
AllowRetry	0 or 1	仅在支持重试时存在

9.1.2 REQ 通道

域名称	宽度	说明
TxnID	`TxnID_Width`	每组端口可以有不同的 TxnID 宽度
SrcID	`UpstreamNodeID_Width`	-
TgtID	`DownstreamNodeID_Width`	-
Opcode	1 to 6	宽度由具体的组件分类，及其支持的事务类型决定；总宽度为 6，因不足位数导致的端口转换必须采用零扩展
Size	3	-
Addr	48	-
TagAlias	`TagAlias_Width`	-

域名称	宽度	说明
NS	1	-
Order	2	-
MemAttr	4	-
Excl	1	仅在 Exclusive 事务中使用
ExpCompData	1	-
ExpCompStash		仅在下层预取事务中使用
WayValid	0 or 1, **由 `UWPredit_Enable` 与 `UWPersist_Enable` 决定**	**路预测值**或**路持久化值**是否有效，仅在上下游同时支持**组合路预测**或**路持久化**时存在
Way	0 or `WayIndex_Width`	根据下游缓存结构决定，仅在上下游同时支持**组合路预测**或**路持久化**时存在
TraceTag	1	-
AllowRetry	0 or 1	仅在支持重试时存在

9.1.3 SNP 通道

域名称	宽度	说明
TxnID	`DBID_Width`	-
SrcID	`DownstreamNodeID_Width`	-
TgtID	`UpstreamNodeID_Width`	-
Opcode	0 or 2	对于 Type 2 组件，总是 SnpMakeInvalid；总宽度为 2，因不足位数导致的端口转换必须采用零扩展
Addr	45	-
NS	1	-
TraceTag	1	-

9.1.4 RSP 通道

1. 由下游出发的 RSP 通道，DnRSP

域名称	宽度	说明
TxnID	`TxnID_Width`	-
SrcID	`DownstreamNodeID_Width`	-
TgtID	`UpstreamNodeID_Width`	-
DBID	`DBID_Width`	-
Opcode	0, 2 or 3	宽度由具体的组件分类，及其支持的事务类型决定；总宽度为 3，因不足位数导致的端口转换必须采用零扩展
RespErr	2	-
Resp	3	-

域名称	宽度	说明
CBusy	3	-
WayValid	0 or 1, **由 `UWPersist_Enable` 决定**	**路持久化值**是否有效，仅在上下游同时支持**路持久化**时存在
Way	0 or `WayIndex_Width`	宽度由下游缓存结构决定，仅在上下游同时支持**路持久化**时存在
TraceTag	1	-

2. 由上游出发的 RSP 通道，UpRSP

域名称	宽度	说明
TxnID	`DBID_Width`	-
SrcID	`UpstreamNodeID_Width`	-
TgtID	`DownstreamNodeID_Width`	-
Opcode	0 or 1	宽度由具体的组件分类，及其支持的事务类型决定；总宽度为 1，因不足位数导致的端口转换必须采用零扩展
RespErr	2	-
Resp	3	-
TraceTag	1	-

9.1.5 DAT 通道

1. 由下游出发的 DAT 通道，DnDAT

域名称	宽度	说明
TxnID	`TxnID_Width`	-
SrcID	`DownstreamNodeID_Width`	-
TgtID	`UpstreamNodeID_Width`	-
DBID	`DBID_Width`	-
Opcode	0	总是 CompData
RespErr	2	-
Resp	3	-
DataSource	5	-
CBusy	3	-
WayValid	0 or 1, **由 `UWPersist_Enable` 决定**	**路持久化值**是否有效，仅在上下游同时支持**路持久化**时存在
Way	0 or `WayIndex_Width`	宽度由下游缓存结构决定，仅在上下游同时支持**路持久化**时存在
DataID	**log2(512 /** `Data_Width`**)**	-
Data	`Data_Width`	-
TraceTag	1	-

2. 由上游出发的 DAT 通道，UpDAT

域名称	宽度	说明
TxnID	`DBID_Width`	-
SrcID	`UpstreamNodeID_Width`	-
TgtID	`DownstreamNodeID_Width`	-
Opcode	0, 1 or 2	宽度由具体的组件分类，及其支持的事务类型决定；总宽度为 2，因不足位数导致的端口转换必须采用零扩展
RespErr	2	-
Resp	3	-
DataID	log2(512 / `Data_Width`)	-
Data	`Data_Width`	-
BE	32	-
TraceTag	1	-

9.2 域定义

9.2.1 事务标识, TxnID

9.2.2 来源节点标识，SrcID

9.2.3 目标节点标识，TgtID

9.2.4 返回事务标识，DBID

9.2.5 请求/消息类型, Opcode

9.2.5.1 EVT 通道

Opcode[0:0]	请求类型
0x00	Evict
0x01	WriteBackFull

9.2.5.2 REQ 通道

Opcode[5:0]	请求类型
0x00	StashShared
0x01	StashUnique
0x02	ReadNoSnp
0x03	ReadOnce
0x04	ReadShared

Opcode[5:0]	请求类型
0x05	Reserved
0x06	Reserved
0x07	Reserved
0x08	WriteNoSnpPtl
0x09	WriteNoSnpFull
0x0A	WriteUniquePtl
0x0B	WriteUniqueFull
0x0C	CleanShared
0x0D	CleanInvalid
0x0E	MakeInvalid
0x0F	Reserved
0x10	ReadUnique
0x11	Reserved (CleanUnique)
0x12	MakeUnique
0x13	Reserved
0x14	Reserved
0x15	Reserved
0x16	Reserved
0x17	Reserved
0x18	Reserved
0x19	Reserved
0x1A	Reserved
0x1B	Reserved
0x1C	Reserved
0x1D	Reserved
0x1E	Reserved (EvictBack)
0x1F	Reserved (EvictClean)
0x20 - 0x27	AtomicLoad
0x28 - 0x2F	AtomicStore
0x30	AtomicSwap
0x31	AtomicCompare
0x32 - 0x3F	Reserved

Opcode[5:3]	请求类型	Opcode[2:0]	原子操作类型
100	AtomicLoad	000	ADD
101	AtomicStore	001	CLR
		010	EOR
		011	SET
		100	SMAX
		101	SMIN

Opcode[5:3]	请求类型	Opcode[2:0]	原子操作类型
		110	UMAX
		111	UMIN

9.2.5.3 SNP 通道

Opcode[1:0]	请求类型
0x00	SnpMakeInvalid
0x01	SnpToInvalid
0x02	SnpToShared
0x03	SnpToClean

9.2.5.4 RSP 通道

1. 由下游出发的 RSP 通道

Opcode[2:0]	消息类型
0x00	CompStash
0x01	Comp
0x02	DBIDResp
0x03	CompDBIDResp
0x04	CompCMO
0x05	Reserved (DBIDRespOrd)
0x06	Reserved
0x07	Reserved

2. 由上游出发的 RSP 通道

Opcode[0:0]	消息类型
0x00	CompAck
0x01	SnpResp

9.2.5.5 DAT 通道

1. 由下游出发的 DAT 通道

Opcode[0:0]	消息类型
0x00	CompData

2. 由上游出发的 DAT 通道

Opcode[1:0]	消息类型
0x00	NonCopyBackWrData
0x01	Reserved (NCBWrDataCompAck)
0x02	CopyBackWrData
0x03	SnpRespData

9.2.6 事务数据长度, Size

Size[2:0]	数据长度 (Bytes)
0b000	1
0b001	2
0b010	4
0b011	8
0b100	16
0b101	32
0b110	64
0b111	-

9.2.7 事务地址，Addr

9.2.8 别名地址，TagAlias

9.2.9 地址隔离保留，NS

9.2.10 定序属性，Order

9.2.11 内存属性，MemAttr

9.2.12 独占访问标识，Excl

9.2.13 期望 CompData 回复，ExpCompData

9.2.14 期望 CompStash 回复，ExpCompStash

9.2.15 路索引有效，WayValid

9.2.16 路索引，Way

9.2.17 状态回复, Resp

Resp[2:0]	状态类型	说明
0b000	I	
0b001	SC	

Resp[2:0]	状态类型	说明
0b010	UC	
0b011	-	
0b100	I_PD	
0b101	SC_PD	
0b110	UC_PD	
0b111	-	

9.2.18 错误信息，RespErr

9.2.19 数据来源标识，DataSource

9.2.20 下游忙碌信息，CBusy

9.2.21 循迹标识，TraceTag

10 事务结构

10.1 非 Cacheable 读

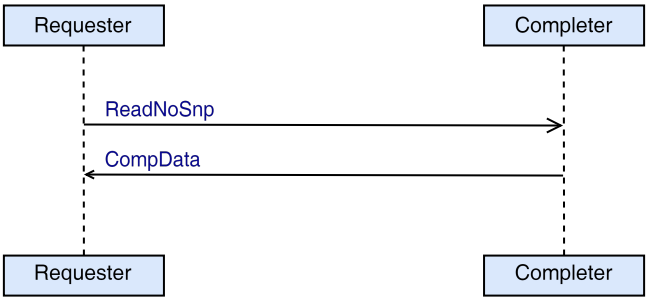


图 10.1: 非 Cacheable 读事务流程

事务流程如下:

- 事务流程从 Requester 向 Completer 发送非 Cacheable 读请求开始。
  初始请求为如下之一:
  - ReadNoSnp
- 在 Requester 收到来自 Completer 的所有 CompData 回复之后，事务流程结束。

这类读事务从非 Cacheable 空间中读取数据。数据允许但不要求是来自于缓存系统中某一时刻最新的数据，允许不经过任何缓存系统，直接到达、来自该地址对应的终点组件。

以下为这类事务的缓存行状态转换及其对应的回复消息:

请求类型	初始状态	最终状态	回复（DnDAT）
ReadNoSnp	I	I	CompData_UC, CompData_I

10.2 Cacheable 读

事务流程如下:

- 事务流程从 Requester 向 Completer 发送 Cacheable 读请求开始。
  初始请求为如下之一:
  - ReadOnce

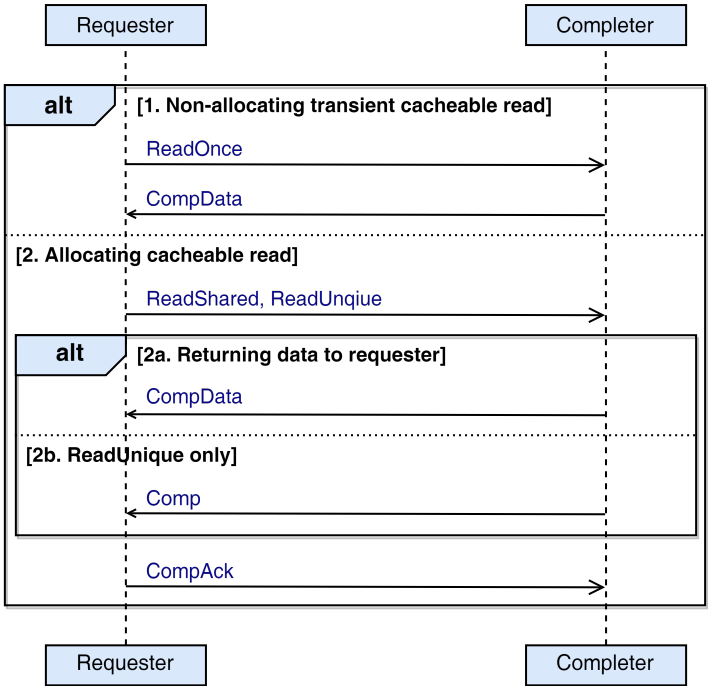


图 10.2: Cacheable 读事务流程

  - ReadShared
  - ReadUnique

· 根据请求类型的不同，Completer 会选择不同的执行路径：

Alt 1. 不进行一致性分配的瞬时 Cacheable 读（Non-allocating transient cacheable read）

这类读事务从 Cacheable 空间中读取数据，即保证可以得到整个缓存系统中某一时刻最新的数据，但在 Requester 处得到数据时不保留任何缓存行状态，在 Completer 处也不会分配 Directory 或 Snoop Filter 条目（如果有）。

· 其初始请求必须为如下之一：

  - ReadOnce

· 在 Requester 收到来自 Completer 的所有 CompData 回复之后，事务流程结束。

Alt 2. 进行一致性分配的 Cacheable 读（Allocating cacheable read）

这类读事务从 Cacheable 空间中读取数据，即保证可以得到整个缓存系统中最新的数据。在 Requester 处得到数据时保留缓存行状态、分配缓存行，在 Completer 处可能需要分配 Directory 或 Snoop Filter 条目（如果有）。

· 其初始请求必须为如下之一：

  - ReadShared
  - ReadUnique

· 根据请求类型的不同，Completer 会选择不同的执行路径：

  - **Alt 2a. Completer 以 CompData 返回数据**
    对于 ReadShared，Completer 总是使用 CompData 返回数据。

对于 ReadUnique，若 Requester 的对应缓存行在 Completer 返回 CompData 或 Comp 之前因 Snoop 而失去了有效的数据，或 Requester 不持有该缓存行的有效数据和有效一致性状态，Completer 必须使用 CompData 返回数据。若 Requester 持有该缓存行的有效数据，允许但不建议 Completer 返回 CompData。

- **Alt 2b. Completer 以 Comp 返回状态并不携带数据**
  仅 ReadUnique 时 Completer 允许以 Comp 返回状态并不携带数据。
  对于 ReadUnique，若 Requester 持有该缓存行的有效数据，允许且建议 Completer 返回 Comp。
  对于 ReadUnique，若 ExpCompData 被置为有效，则不允许使用这一路径。

· Requester 向 Completer 发送 CompAck，事务流程结束。
在 Requester 收到任意的 CompData 或 Comp 之前，不允许发送 CompAck。

以下为这类事务的缓存行状态转换及其对应的回复消息：

请求类型	初始状态	最终状态	回复（DnDAT/DnRSP）
ReadOnce	I	I	CompData_UC, CompData_I
ReadShared	I, SC	SC	CompData_SC
		UC	CompData_UC
		UD	CompData_UD_PD
ReadUnique	I	UC	CompData_UC
		UD	CompData_UD_PD
	SC, UC	UC	CompData_UC, Comp_UC (ExpCompData = 0)
		UD	CompData_UD_PD
	UD	UD	CompData_UC*, Comp_UC* (ExpCompData = 0), CompData_UD_PD*

* **注：** 在 Upstream 持有 UD 而发起 ReadUnique 时，任何从 Downstream 返回的数据都是过时的数据，Upstream 应当丢弃或作适当的数据合并

10.3 缓存行踢出

事务流程如下：

· 事务流程从 Requester 向 Completer 发送缓存行踢出请求开始。
初始请求为如下之一：

  - Evict

· 在 Requester 收到来自 Completer 的 Comp 消息后，事务流程结束。

以下为这类事务的缓存行状态转换及其对应的回复消息：

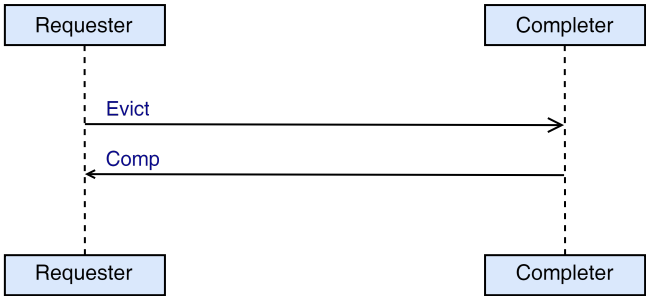


图 10.3: 缓存行踢出事务流程

请求类型	初始状态	最终状态	回复（DnRSP）
Evict	I	I	Comp

10.4 缓存行写回

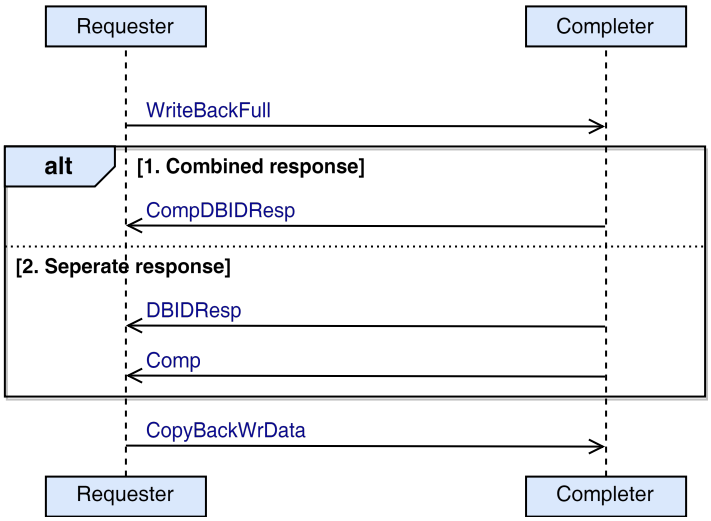


图 10.4: 缓存行写回事务流程

事务流程如下:

- 事务流程从 Requester 向 Completer 发送缓存行写回请求开始。
  初始请求为如下之一:
  - WriteBackFull

Alt 1. 组合回复（CompDBIDResp）

- 在 Requester 收到来自 Completer 的 CompDBIDResp 消息后，开始向 Completer 通过 CopyBackWrData 发送写回数据。
  在 Requester 收到 CompDBIDResp 之前，不允许发送 CopyBackWrData。
  若在 Requester 决定向 Completer 发送缓存行请求之后，收到了来自 Completer 的同物理地址的 Snoop 请求，则必须等待到收到来自 Completer 的 CompDBIDResp 后再对 Snoop 请求进行回复。
- 在 Requester 发送了所有的 CopyBackWrData 后，事务流程结束。

Alt 2. 分步回复（DBIDResp 与 Comp）

- 在 Requester 收到来自 Completer 的 DBIDResp 消息后，开始向 Completer 通过 CopyBackWrData 发送写回数据。
  在 Requester 收到 DBIDResp 之前，不允许发送 CopyBackWrData。

· 在 Requester 发送了所有的 CopyBackWrData 且收到 Comp 回复后，事务流程结束。

以下为这类事务的缓存行状态转换及其对应的回复消息：

请求类型	初始状态	最终状态	数据回复（UpDAT）	回复（DnRSP）
WriteBackFull	UD	I	CopyBackWrData_UD_PD	DBIDResp + Comp, CompDBIDResp

10.5 无数据权限转移

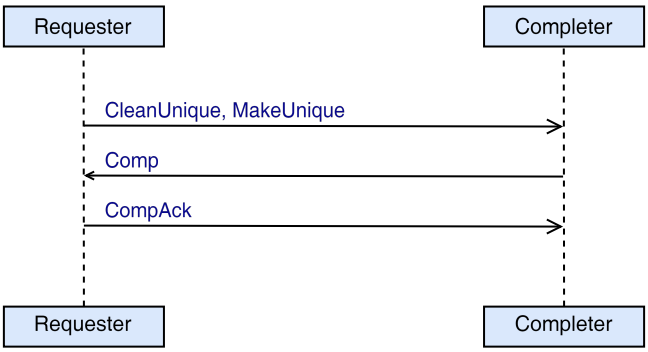


图 10.5: 无数据权限转移

事务流程如下：

· 事务流程从 Requester 向 Completer 发送无数据权限转移请求开始。
初始请求为如下之一：

  – CleanUnique
  – MakeUnique

· Completer 向 Requester 发送 Comp 消息，并携带缓存行的一致性状态信息。

· Requester 向 Completer 发送 CompAck 消息后，事务流程结束。在 Requester 收到 Comp 之前，不允许发送 CompAck。

以下为这类事务的缓存行状态转换及其对应的回复消息：

请求类型	初始状态	最终状态	回复（DnRSP）
MakeUnique	I	UD	Comp_UC

10.6 缓存行维护（CMO）

事务流程如下：

· 事务流程从 Requester 向 Completer 发送无数据权限转移请求开始。
初始请求为如下之一：

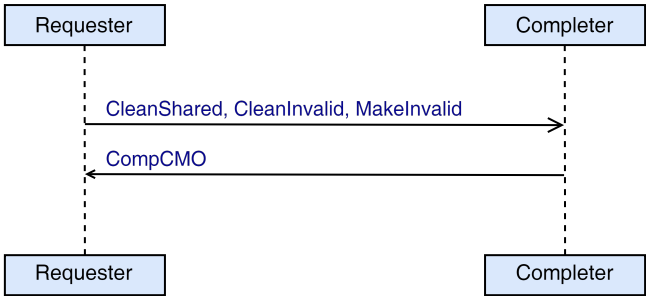


图 10.6: 缓存行维护事务流程

  - CleanShared
  - CleanInvalid
  - MakeInvalid
- Completer 向 Requester 发送 CompCMO 消息后，事务流程结束。只有在 Completer 能够保证所有的 I/O Coherent 组件都能够观察到 CMO 的结果时，才可以返回 CompCMO。允许但不建议 Completer 等到 CMO 结果对所有的组件可见（包括 Non-coherent 组件 [**注**]）。
  需要注意 CompCMO 与 Comp 的语义不完全相同。

  注： RISC-V Zicbom 扩展中要求 `cbo.inval`、`cbo.flush`、`cbo.clean` 指令需要保证对于 Non-coherent 设备的可见性顺序，但 Compact CHI 架构不保证这一点，并且不打算规定这一种保证。此种保证对 Compact CHI 之外的总线，通常为更加下游的总线（例如 AMBA CHI）会产生过于严格的功能性依赖、限制，严重影响总线协议的兼容性。为了保证总线的实现能够满足 RISC-V Zicbom 扩展的要求，请注意在 SoC 层面进行额外的适配、改动。

以下为这类事务的缓存行状态转换及其对应的回复消息：

请求类型	初始状态	最终状态	回复（DnRSP）
CleanShared	I	I	CompCMO*
	SC	SC	CompCMO*
	UC	UC	CompCMO*
	UD	UC*	CompCMO*
CleanInvalid	I, SC, UC, UD	I*	CompCMO*
MakeInvalid	I, SC, UC, UD	I*	CompCMO*

* **注：**此处 CompCMO 并不实际导致对 Requester 缓存行状态的转换，在 Requester 收到 CompCMO 时必须保持其最终状态；对于所有的 CMO 类请求来说，若在 Completer 收到请求时 Requester 的缓存行状态尚不符合最终状态要求，Completer 必须通过嵌套的 Snoop 请求来更新 Requester 目前持有的缓存行状态，并且在发送 CompCMO 之前不再响应任何由其它请求引起的、不符合 CMO 请求最终缓存行状态要求的操作；Completer 允许但不建议在 Requester 的缓存行状态已经符合要求的情况下，发起嵌套的 Snoop 请求

10.7 跨层预取

事务流程如下：

- 事务流程从 Requester 向 Completer 发送跨层预取请求开始。
  初始请求为如下之一：
  - StashShared
  - StashUnique
- 如果初始请求中的 ExpCompStash 被置为了有效，Completer 需要向 Requester 提供 CompStash 消息

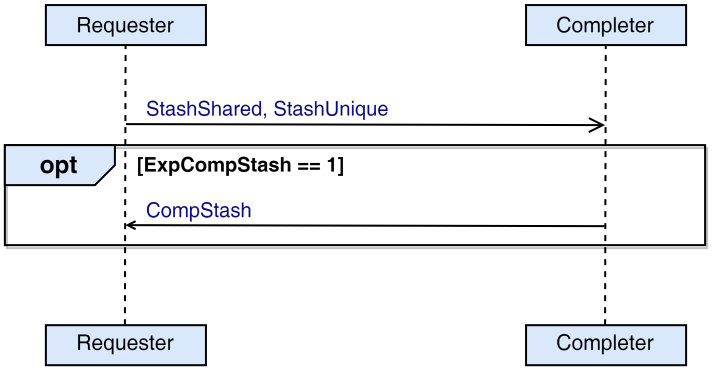


图 10.7: 跨层预取事务流程

以下为这类事务的缓存行状态转换及其对应的回复消息：

请求类型	初始状态	最终状态	回复（DnRSP）
StashShared	I	I	CompStash (ExpCompStash=1)
	SC	SC	
	UC	UC	
	UD	UD	
StashUnique	I	I	CompStash (ExpCompStash=1)
	SC	SC	
	UC	UC	
	UD	UD	

10.8 非 Cacheable 写

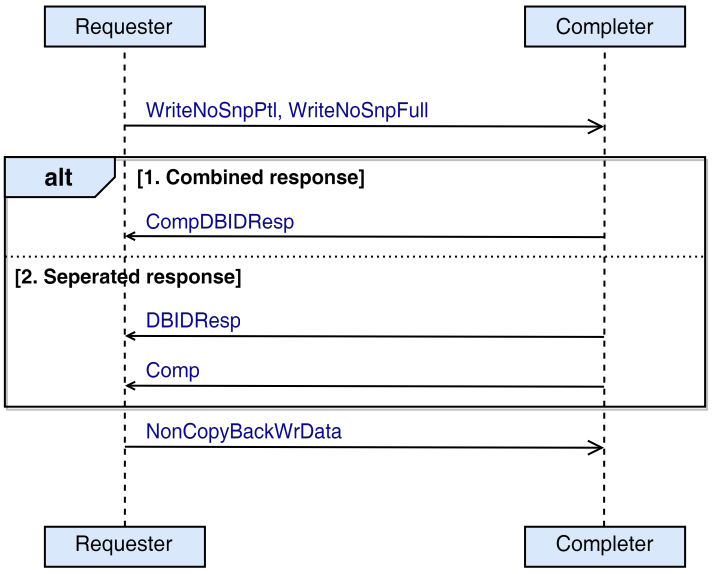


图 10.8: 非 Cacheable 写事务流程

事务流程如下：

- 事务流程从 Requester 向 Completer 发送非 Cacheable 写请求开始。
  初始请求为如下之一：
  - WriteNoSnpPtl
  - WriteNoSnpFull
- 根据 Completer 的实现，Completer 可以选择不同的执行路径：

* Completer 使用 DBIDResp 向 Requester 回复 DBID 信息

* Completer 使用 Comp 回复 Requester
  Completer 允许但不要求在回复 Comp 之前等待来自 Requester 的数据。

以下为这类事务的缓存行状态转换及其对应的回复消息：

请求类型	初始状态	最终状态	数据回复（UpDAT）	回复（DnRSP）
WriteNoSnpPtl	I	I	NonCopyBackWrData	DBIDResp + Comp, CompDBIDResp
WriteNoSnpFull	I	I	NonCopyBackWrData	DBIDResp + Comp, CompDBIDResp

10.9 Cacheable 写

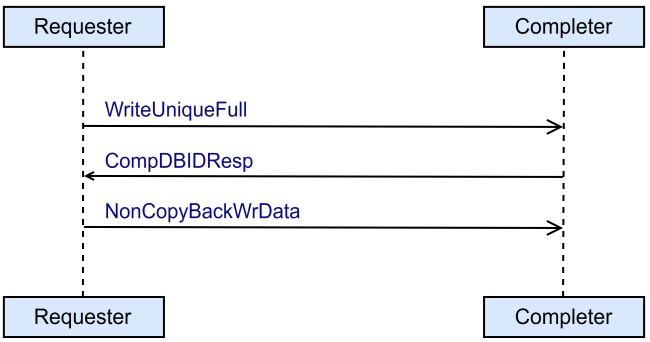


图 10.9: Cacheable 写事务流程

事务流程如下：

· 事务流程从 Requester 向 Completer 发送 Cacheable 写或缓存行写回请求开始。
  初始请求为如下之一：

  – WriteUniqueFull

这类写事务向 Cacheable 空间中写入数据，且在其符合缓存一致性协议时，能够保证其写入的数据在某个时刻是整个缓存系统中最新的。直写意味着 Requester 在发起请求前自身不持有有效的一致性状态，且不造成自身缓存行状态的转移。

· Completer 向 Requester 发送 CompDBIDResp，其中包含 DBID 信息与 Comp 语义。

· Requester 向 Completer 通过 NonCopyBackWrData 消息发送所有的数据后，事务结束。Requester 必须在收到 CompDBIDResp 之后才可以发送 NonCopyBackWrData。

以下为这类事务的缓存行状态转换及其对应的回复消息：

请求类型	初始状态	最终状态	数据回复（UpDAT）	回复（DnRSP）
WriteUniqueFull	I	I	NonCopyBackWrData	DBIDResp + Comp, CompDBIDResp

10.10 原子操作

TBD

10.11 Snoop

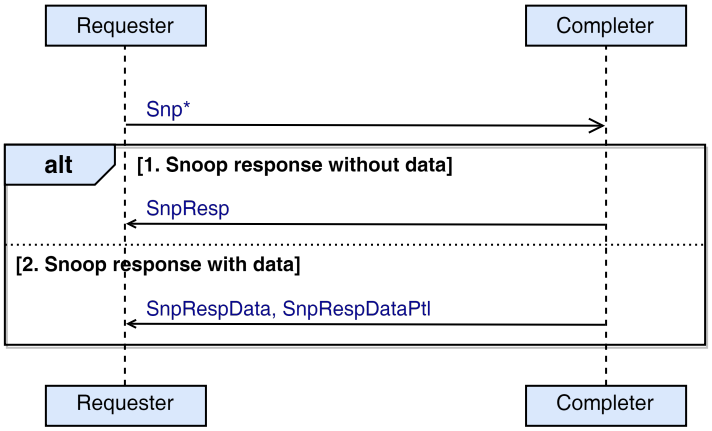


图 10.10: Snoop 事务流程

事务流程如下：

- 事务流程从 Requester 向 Completer 发送 Snoop 请求开始。
  初始请求为如下之一：
  - SnpMakeInvalid
  - SnpToInvalid
  - SnpToShared
  - SnpToClean
- 根据事务类型以及 Completer 持有的缓存行一致性状态，Completer 会选择不同的执行路径：

Alt 1. 向 Requester 返回状态但不携带数据

- Completer 向 Requester 发送 SnpResp 后，事务结束。

Alt 2. 向 Requester 返回状态并携带数据

- Completer 向 Requester 发送所有 SnpRespData 后，事务结束。

具体执行路径的选择规则请参阅下表中 Snoop 请求的缓存行一致性状态转换规则。

请求类型	起始状态	回复前状态 *	最终状态	回复 （UpRSP/UpDAT）
SnpMakeInvalid	UD	UD, UC, SC, I	I	SnpResp_I
	UC	UC, SC, I	I	SnpResp_I
	SC	SC, I	I	SnpResp_I
	I	I	I	SnpResp_I
SnpToInvalid	UD	UD	I	SnpRespData_I_PD
		UC, SC	I	SnpRespData_I
		UC, SC, I	I	SnpResp_I
	UC	UC, SC	I	SnpRespData_I
		UC, SC, I	I	SnpResp_I
	SC	SC	I	SnpRespData_I
		SC, I	I	SnpResp_I
	I	I	I	SnpResp_I
SnpToShared	UD	UD	SC	SnpRespData_SC_PD
			I	SnpRespData_I_PD
	UD, UC	UC, SC	SC	SnpRespData_SC
				SnpResp_SC
			I	SnpRespData_I
				SnpResp_I
		I	I	SnpResp_I
	SC	SC	SC	SnpRespData_SC
				SnpResp_SC
			I	SnpRespData_I
				SnpResp_I
		I	I	SnpResp_I
	I	I	I	SnpResp_I
SnpToClean	UD	UD	UD, UC	SnpRespData_UD_PD
			SC	SnpRespData_SC_PD
			I	SnpRespData_I_PD
	UD, UC	UC	UC	SnpRespData_UC
				SnpResp_UC
			SC	SnpRespData_SC
				SnpResp_SC
			I	SnpRespData_I
				SnpResp_I
	UD, UC, SC	SC	SC	SnpRespData_SC
				SnpResp_SC
			I	SnpRespData_I
				SnpResp_I
	UD, UC, SC, I	I	I	SnpResp_I

* **注：** 回复前状态标注的意义在于，在 Snoop 请求的回复状态确定之前，可能会发生嵌套的踢出、写回，此时缓存行状态亦会嵌套地改变

11 上游路持久化

上游路持久化（UWPersist, Upstream Way Persistence）主要用于 Inclusive 缓存系统中的上游 L1 Cache 替换踢出流程优化, 不适用于 Exclusive、Non-Inclusive Non-Exclusive 的缓存系统。此特性主要通过 L1 Cache 对 L2 Cache 中持有该缓存行的路索引进行额外的持久化，加速后续 L1 Cache 对 L2 Cache 发起的缓存行踢出或缓存行写回请求流程，并可能减少 L2 Cache 中对于本地目录、Snoop Filter 等资源组件的访问次数。

此特性通过 EVT、DnRSP、DnDAT 通道中的 Way、WayValid 域。当对应消息的 WayValid 有效时，Way 必须携带有效的路索引，并且 Way 的值必须是准确的。下游不要求实现对 Way 有效性的检查，并且可以总是认为收到的 Way 与本地目录中实际存储的、该缓存行的实际路索引是一致的。若 Way 值存在问题，则其后续结果是未定义的行为。

可以使用上游路持久化的请求类型，及其传递路索引的消息类型如下所示。

对于支持上游路持久化的 L2 Cache，其 WayValid 必须在以下情况中总是有效，而 L1 Cache 允许丢弃 L2 Cache 提供的路索引值。对上游可能产生可被持久化的路索引的消息类型如下表所示，此时 L2 Cache 会将该缓存行所在的路索引传递给 L1 Cache。

请求类型	REQ	DnRSP (Comp)	DnDAT (CompData)	UpRSP (CompAck)
MakeUnique	-	Y	-	-
ReadShared	-	-	Y	-
ReadUnique	-	Y*	Y	-

* **注：** 在 L1 Cache 处于 S/UC/UD 状态下发生 ReadUnique 有可能通过 Comp 直接得到权限，此时 L1 Cache 已经持有该缓存行，并且可能已经持久化路索引。在这种情况下 L1 Cache 允许但不要求重新写入 Comp 携带的 Way 值，但在 L2 Cache 明确支持路持久化时 WayValid 必须有效并携带合法的 Way 值。

对于不支持上游路持久化的 L1 Cache，其 WayValid 必须是无效的。对于支持上游路持久化的 L1 Cache, 亦允许使 WayValid 无效，具体情况由实现决定。对下游可能发送被持久化的路索引的消息类型如下表所示，此时 L1 Cache 会将该缓存行所在的路索引传递给 L2 Cache。

请求类型	EVT	DnRSP (Comp*)	UpDAT (CopyBackWrData)
Evict	Y	-	-
WriteBackFull	Y	-	-

上游路持久化的可应用场景及其可达成的优化效果如下例:

· L1 Cache 的一个 `load` 操作发生缺失，向 L2 Cache 发起 ReadShared 请求

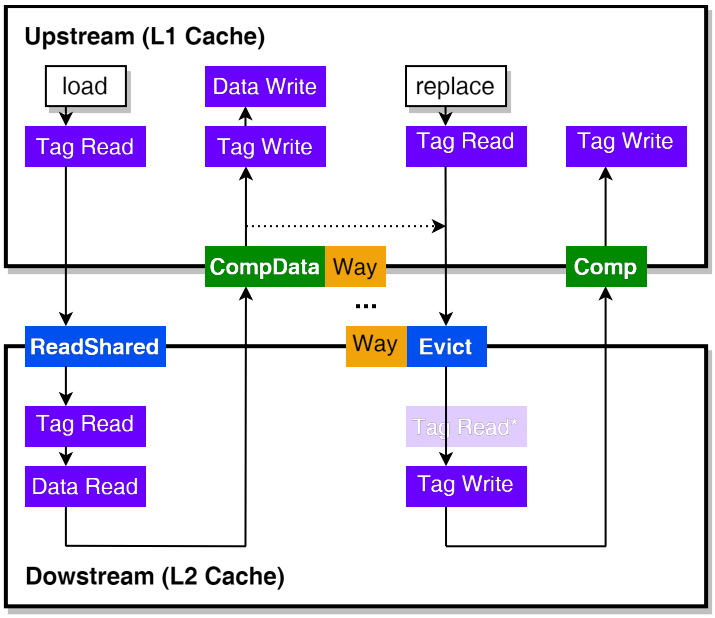


图 11.1: 上游路持久化实现例

- L2 Cache 收到 ReadShared，读取 Tag Array 与 Data Array 后命中，随即通过 CompData 将读权限与数据返回给 L1 Cache，**并且通过 Way 域携带实际命中的路索引值，置 WayValid 为有效**
- L1 Cache 收到 CompData 时，**将 Way 域携带的路索引值写入 Tag Array，以此完成路持久化**
- L1 Cache 发生替换，需要踢出该缓存行，在读取 Tag Array 后向 L2 Cache 发起 Evict 请求，**并携带有效的 WayValid 与 Way 值**
- L2 Cache 收到 Evict **及其携带的有效 Way，直接更新 Tag Array / Snoop Filter 中对应路的值，而不再重新读取 Tag Array 来查询其对应的路索引**
- L2 Cache 向 L1 Cache 回复 Comp
- L1 Cache 更新 Tag Array

12　上游路预测

（UWPredict，Upstream Way Prediction）

第四部分

链路层

13 时钟与初始状态

13.1 时钟

本标准不定义、不指定某种时钟架构，并允许使用多个时钟域。在使用多时钟域、跨时钟域桥时，需要额外注意 Credit 的分配数量、形式来保证系统能够达到最大的吞吐量。

13.2 复位

本标准不定义、不指定某种复位架构。

13.3 初始化

复位期间以下链路层信号必须全部置为低（不论高有效还是低有效的信号）：

- **PDCRD*GNT**
- **PDCRD*RCLREQ**
- **PDCRD*RCLACCEPT**
- **PDCRD*RCLDENY**
- **PSCRD*GNT**
- **PRETRYV**
- **LQOSLIFT**
- **LQOSDROP**
- **TX**FLITV**

其它信号可以是任意值。

14 协议资源流控

本节内容将介绍链路层的基于 Credit 的协议资源流控（下简写 P-Credit）机制。每个 Credit 由 Completer 向 Requester 通过单周期的 ***CRD*GRANT** 高有效信号发送。

在 Compact CHI 规范中，链路层的 P-Credit 被分为动态 Credit 与静态 Credit 两种。其中动态 P-Credit 在到达 Requester 之后，可以再被 Completer 收回。同时动态 P-Credit 允许出现 Requester 中持有数量与 Completer 中空闲资源数量不一样的情况。其具体的作用与区别将会在本节内容中详细介绍。

P-Credit 的传递允许在路途中跨过 Buffer，或跨时钟域，但本标准所定义的机制不鼓励这么做。通常来说，Credit 的管理、交互是端到端的（任意中间路径上的 Buffer 也算作若干个端口），本标准中定义的 P-Credit 机制也允许这种形式，Retry 机制能够保证协议本身不会出现死锁、信息丢失，但可能会导致 Requester 无法精确获得 Completer 内的资源可用情况，这对于通道带宽利用率与总体延迟可能是有害的，可能违背本机制原本的设计目标。本标准中 P-Credit 的根本设计考虑在于，Compact CHI 主要用于直接连接若干个同时钟域且物理位置相近的组件，主要面向于结构上大致点对点的场景，而不考虑复杂的网络层拓扑，例如 L1 Cache 与 L2 Cache 之间的一致性连接。

> **注：**本节中的 P-Credit 与 AMBA CHI 协议中的定义不同，是两种完全不同的机制，请注意不要混淆。AMBA CHI 中的 P-Credit 定义在协议层，而 Compact CHI 中的 P-Credit 定义在链路层，并允许协议层的一些定义和共识与链路层耦合，以实现精细化的流量控制。

14.1 P-Credit 流控

在 Compact CHI 规范中，Completer 通过告知 Requester 其内部的资源可用情况来实现对于 Requester 的流量控制。Completer 使用动态与静态的 P-Credit 来向 Requester 同步其可用的协议资源。Completer 内的资源不需要全部都对每一个 Requester 可见。若某个 Completer 在某条链路上存在多个 Requester，则需要在 Completer 动态地为每个 Requester 仲裁、分配可见的资源以达到精细化的流量控制。

在 Compact CHI 规范中，各种不同类型的事务会需要占用不同的 Completer 资源。每种事务具体需要什么样的 Completer 资源是由具体**实现定义的**，本规范中不作任何要求。在具体实现能够保证其正确性的时候，亦允许某种事务在链路层被定义为不需要任何 Completer 资源，但这种事务请求是不允许被重试的，即使首次发出请求的 **AllowRetry** 域必须为有效。每一种 Completer 资源对应一种 P-Credit 类型，其对应的信号名如下：

- **P*CRD{n}***

每种 Completer 资源对应的 P-Credit 使用不同编号的信号进行区分，例如对于第 0 种类型的资源 Credit 的信号名如下：

- **P*CRD0***

亦允许直接使用 Completer 资源的名称、缩写来编写信号名，需要注意各个信号名之间不能有混淆，或尽量不要出现过长的信号名。对于不同的端口，允许使用不同的 P-Credit 定义。

对于 Completer 资源类型与其 P-Credit 信号组的名称，Requester 与 Completer 之间必须要有一致的定义。此种定义的共识是在设计时就完成的，在 Compact CHI 中建议硬编码到 Requester 与 Completer 各自的逻辑中。本标准不定义、提供任何在 Requester 与 Completer 之间同步这种共识的额外方法，但也不禁止具体设计使用软件固件控制等其他方法同步、修改这种共识。

14.1.1 动态 P-Credit 及其同步

动态 P-Credit 使用独立的信号通道来传递 Credit 与同步信息，各个信号如图 4. 形式在 Requester 与 Completer 之间连接。

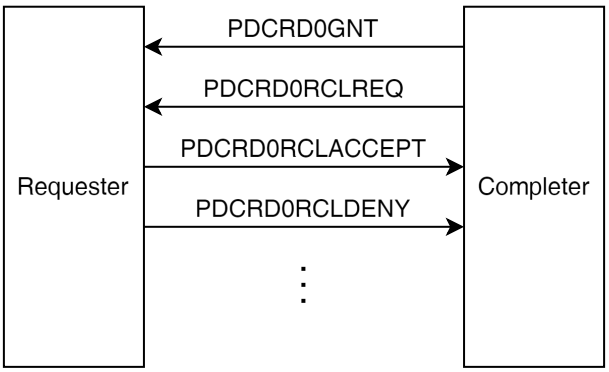


图 14.1: 动态 P-Credit 的传递与同步信号

动态资源是由 Completer 预先为 Requester 动态预留的资源。动态资源以动态 P-Credit 的形式从 Completer 传递到 Requester。同时，允许 Completer 通知 Requester 收回对应资源。Requester 亦可以拒绝这种收回的通知，其会导致 Requester 的资源 Credit 与 Completer 失去同步，后续的事务请求可能被重试。其对应的信号名与说明如表 4.。

表 14.1: 动态 P-Credit 的传递与同步信号列表

信号名	信号源	功能说明
PDCRD*GNT	Completer	高有效，表示当前周期正在向 Requester 发送有效的动态 P-Credit，且 Completer 已认为 Requester 持有该动态 P-Credit
PDCRD*RCLREQ	Completer	高有效，表示当前周期正在向 Requester 发送有效的动态 P-Credit 收回请求
PDCRD*RCLACCEPT	Requester	高有效，表示当前周期 Requester 接受了来自 Completer 的 P-Credit 的收回请求
PDCRD*RCLDENY	Requester	高有效，表示当前周期 Requester 拒绝了来自 Completer 的 P-Credit 的收回请求

PDCRD*GNT 使用一个周期的高有效信号来表示 Completer 向 Requester 发送了一个对应类型的动态 P-Credit。

Requester 在首次发出事务请求时，必须确保自己持有足够的动态 P-Credit，否则出现**未定义的行为**。请求在离开 Requester 时，对应的动态 P-Credit 在 Requester 内应当被立即消耗。

动态资源的 “动态性” 在于 Completer 可以在任何时候重新同步已经发送给 Requester 的动态 P-Credit。此种动态的设计主要出于 Completer 资源能够被充分利用的考虑。

例如某个 Completer 端口对应着多个上游的 Requester 端口，那么 Completer 需要预先将一些动态的 P-Credit 分别发送到各个 Requester 上。在某一时刻，有可能存在某一个 Requester 在一段时间内需要大量的带宽与资源，而其它 Requester 几乎不产生事务的情况。此时，Completer 就可以通知其它 Requester 其预先分配的动态资源需要被回收，并将这些优先资源分配给需要大量资源的那一个 Requester，尽可能避免其它 Requester 的请求在这段时间内出现重试，以节省 TXREQ 的带宽并降低总线的动态功耗。

PDCRD*RCLREQ 使用一个周期的高有效信号来表示 Completer 向 Requester 发送了一个对应类型的动态 P-Credit 的收回请求。动态 P-Credit 的收回请求应当被作为一种 Completer 向 Requester 同步动态资源分配情况的一种 Hint。Requester 可以拒绝收回请求并认为自己仍然持有该动态 P-Credit，且此时 Completer 应当需要精确跟踪每一个 Requester 内的动态 P-Credit 数量。在 Requester 收到 **PDCRD*RCLREQ** 后，必须在较短的时间内回复 **PDCRD*RCLACCEPT** 或 **PDCRD*RCLDENY** 以告知 Completer 同步情况。通道内出现的 **PDCRD*RCLACCEPT** 与 **PDCRD*RCLDENY** 的数量之和必须和 **PDCRD*RCLREQ** 的数量完全相同，对于回复顺序与其具体对应哪一个周期到达 Requester 的动态 P-Credit 没有约束，但在数量上必须是严格对应的。

Requester 一旦决定返回 **PDCRD*RCLACCEPT**，就必须减少对应动态 P-Credit 的持有数量，且此时必须保证 Requester 正持有对应可返回的动态 P-Credit。

Requester 收到的 **PDCRD*RCLREQ** 是一种 Hint,但返回的 **PDCRD** 与 **PDCRD** 必须精确反应 Requester 所作出的动作。

对于 Requester 允许但不建议只回复 **PDCRD*RCLDENY**，此种实现下 Retry 机制依然能够保证事务的正常推进。具体请见后文。

PDCRD*RCLREQ 与 Requester 的动态 P-Credit 消耗可能存在竞争或失去同步的情况。Requester 允许在请求真正发出前的相当一段时间内就将某个动态 P-Credit 分配给该请求，并认为它已经被消耗。Requester 有可能在认为自己已经不持有任何动态 P-Credit 的情况下收到 **PCRD*RCLREQ**，此时必须回复 **PRD*RCLDENY**。

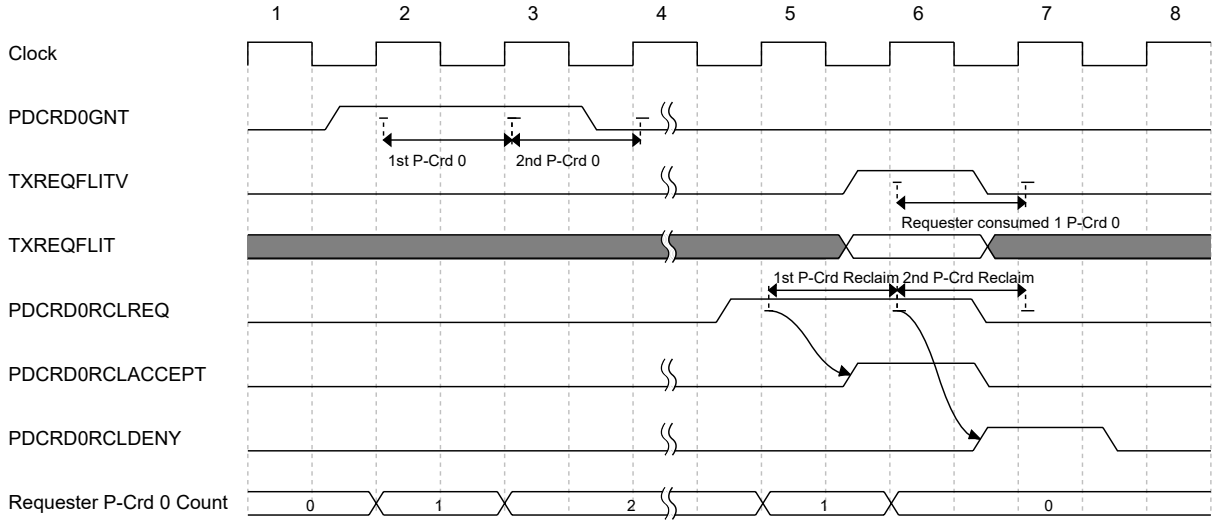


图 14.2: 动态 P-Credit 收回时可能的竞争情况时序示例图

以图 4. 为例。Requester 在 2 号与 3 号上升沿时收到了来自 Completer 的第 0 类动态 P-Credit，即两个周期的有效 **PDCRD0GNT**。可见，在 3 号上升沿之后，该 Requester 就持有了两个第 0 类的动态 P-Credit。经过一段时间后，Completer 在 5 号与 6 号上升沿向 Requester 发送了第 0 类动态 P-Credit 的收回请求，即两个周期的有效 **PDCRD0RCLREQ**。在第 5 号上升沿的收回请求被接受，在 Requester 内部持有的第 0 类因此减少了 1 个，从 2 个变为 1 个，并在下一个周期向 Completer 发回了 **PDCRD0RCLACCEPT**。在第 6 号上升沿时，Requester 向 Completer 发送了一个请求，且这个请求消耗了 Requester 内所持有的最后一个第 0 类动态

P-Credit。此时 Requester 发送的请求就与来自 Completer 的动态 P-Credit 收回请求发生了竞争。Requester 选择了将事务请求发出，就意味着动态 P-Credit 已经分配给发出的事务请求，所以 Requester 已经不再持有第 0 类动态 P-Credit 了。故第 6 号上升沿的收回请求被拒绝，在下一周期向 Completer 发回了 **PDCRD0RCLDENY**。由于 Credit 的传递在 Requester 与 Completer 之间允许且通常存在延迟，故这种竞争是无法避免的，并且在 6 号上升沿发出的请求通常最终会被 Completer 发回重试。

14.1.2 静态 P-Credit

静态 P-Credit 使用独立的信号通道来传递 Credit，各个信号如图 4. 形式在 Requester 与 Completer 之间连接。

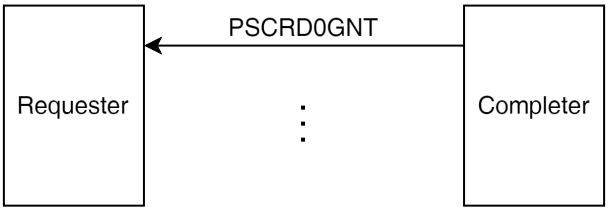


图 14.3: 静态 P-Credit 的传递信号

静态资源是 Completer 将 Requester 的某个请求重试后,在 Completer 内静态预先分配的资源。Requester 在被重试后第二次发出事务请求时，必须确保自己持有足够的静态 P-Credit，否则出现**未定义的行为**。请求在离开 Requester 时，对应的静态 P-Credit 在 Requester 内应当被立即消耗。其对应的信号名与说明如表 4.。

表 14.2: 静态 P-Credit

信号名	信号源	功能说明
PSCRD*GNT	Completer	高有效，表示当前周期正在向 Requester 发送有效的静态 P-Credit

静态资源的 “静态性” 在于其被分配为某个 Requester 之后，就不能再被 Completer 收回或重新分配给其它 Requester，直到被分配的 Requester 将其消耗。

静态 P-Credit 主要被用于重试的事务。对于不使用动态 P-Credit 且不支持 Retry 机制的通道，也可以仅使用静态 P-Credit 进行流控。对于保证永远可以接受消息并完整处理事务的端口，也可以既不使用动态 P-Credit，也不使用静态 P-Credit。

14.2 基于资源的请求 Retry

当两个节点之间的动态 P-Credit 数量出现不同步时，就需要 Retry 机制来告知 Requester 请求暂时无法被完成，且 Completer 必须保证请求流程在一段时间内总能够获得必须的资源并完成。

静态 P-Credit 数量在两个节点之间不允许出现不同步。

任意请求都最多只可以被重试一次。请求首次离开 Requester 时，其中的 **AllowRetry** 域必须被置为高。请求被重试后，再一次离开 Requester 时，**AllowRetry** 域必须被置为低，以指示该请求不能再被重试，且指示 Completer 使用预先静态分配的资源。

对于请求重试中，Requester 第二次发出消息内各个域的值的约束与要求见 {reserved} 章节。

当 Requester 已经知晓自己不再持有某种请求所必需的动态 P-Credit 时，Requester 不允许向 Completer 发起相应的请求。

Retry 机制所能够容忍的最大同时被重试请求数量为 Requester 能够持有的动态 P-Credit 的最大总数量，以此来限制 Completer 需要追踪的重试请求数量与静态分配的资源。通常来说，达到重试请求最大数量的原因是所有来自 Completer 的动态 P-Credit 召回请求都被 Requester 拒绝了。

Completer 与 Requester 中记录的动态 P-Credit 数量出现不同步是一种竞争情况，此时必须使用 Retry 机制保证协议流程的正常推进，并使得 Completer 为其静态地分配资源并保证其能够尽快继续执行。最典型场景为 Completer 向 Requester 发回的动态 P-Credit 召回请求被拒绝。在 Requester 尚未持有对应动态 P-Credit 时，不允许向 Completer 发送对应请求。若该请求已经被重试过，那么就必须等待到 Requester 持有相应的静态 P-Credit 后才能被发送。

Retry 机制应当在大部分情况下被作为一种维持请求流程不会丢失的保证性机制。Requester 应当利用 P-Credit 的所有相关机制来判断请求是否能够被 Completer 接收并处理，并依此决定某请求是否适合立刻发送，而不建议总是依赖 Retry 机制。Requester 应当在有足够信息时，或资源信息被及时反馈时，尽可能避免使用 Retry 机制。此种约定亦是为了尽可能保证 TXREQ 通道的最大带宽。

Retry 机制使用独立的通道信号来传递请求被重试的消息，以节省 RXRSP 通道的带宽：

- **PRETRYV**
- **PRETRYTXNID**
- **PRETRYCRD***

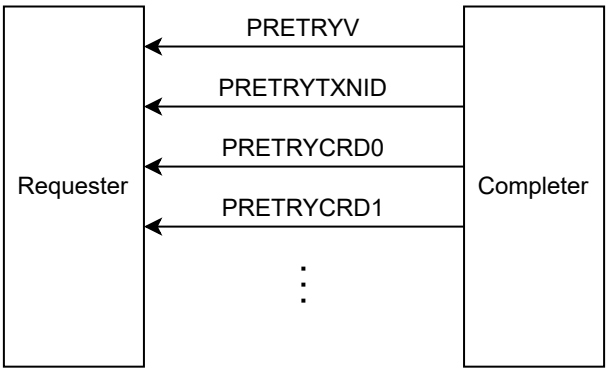


图 14.4: P-Credit Retry 机制信号

各个信号以如图 4.1 形式在 Requester 与 Completer 之间连接，其说明见表 4.1：

表 14.3: P-Credit Retry 机制信号列表

信号名	信号源	功能说明
PRETRYV	Completer	高有效，表示当前周期正在向 Requester 传回有效的 Retry 消息
PRETRYTXNID	Completer	表示当前被重试请求的 TxnID
PRETRYCRD*	Completer	表示当前被重试请求需要重新获取的 Completer 资源类型

图 4.2 为 Completer 传回有效 Retry 消息时的时序示例图。每一个有效的 Retry 消息至少且只会持续一个时钟周期，且单向的有效信号 **PRETRYV** 意味着 Requester 必须在任何时候都能够接收来自 Completer 的 Retry 消息，否则 Retry 消息就有可能在总线上丢失。

在 **PRETRYV** 为低，即通道内没有有效的 Retry 消息时，其它信号允许是任意的值。

当 **PRETRYV** 为高，即通道内正在发送有效的 Retry 消息时：**PRETRYTXNID** 必须为被重试的请求的 TxnID；**PRETRYCRD*** 必须对应为请求所需要重新获取的 Completer 资源类型。若 **PRETRYTXNID** 的 TxnID 无法对应任何 Requester 已发出的请求，则进入**未定义的行为**。若 **PRETRYCRD*** 无法精确对应请求所需要重新获取的 Completer 资源类型，则进入**实现定义的行为**，根据具体实现可能出现死锁、活锁或导致其它请求的不必要的 Retry 流程。其中 **PRETRYCRD*** 全为低是一种不允许的情况，此种 Retry 流程可能会导致被重试请求的数量超出能容忍的数量。

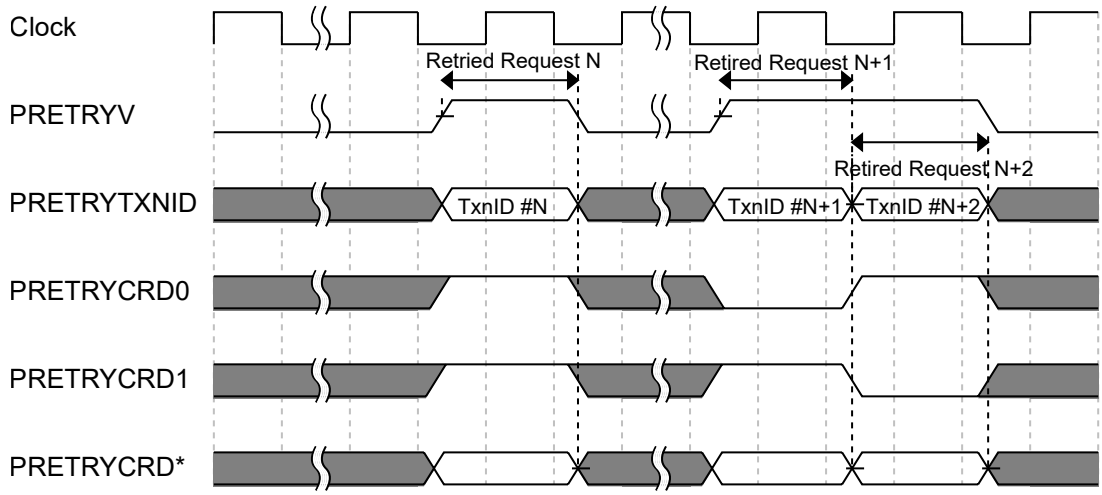


图 14.5: P-Credit Retry 消息信号时序示例图

以图 4.2 为例，Completer 在不同时间向 Requester 发送了三个 Retry 消息。Retry 消息允许背靠背地发送，每一个单独的 Retry 消息只会且只允许持续一个周期。

以图 4.2 中第 N 个重试消息为例，假设该请求需要第 0 类资源与第 1 类资源。此时 Completer 内记录 Requester 不持有任何第 0 类资源和第 1 类资源的 P-Credit，且无法立刻为该请求分配任何的第 0 类资源与第 1 类资源，决定通知 Requester 该请求被重试。**PRETRYTXNID** 的值对应第 N 个被重试的请求的 TxnID。**PRETRYCRD0** 为高表示 Requester 需要等待 Completer 发回第 0 类资源的静态 P-Credit。**PRETRYCRD1** 为高表示 Requester 需要等待 Completer 发回第 1 类资源的静态 P-Credit。Requester 在收到全部以上静态 P-Credit 后才可以重发对应的请求。

以图 4.2 中第 N + 1 个重试消息为例，假设该请求需要第 1 类资源。此时 Completer 内记录 Requester 不持有任何第 1 类资源，且无法立刻为该请求分配任何第 1 类资源，决定通知 Requester 该请求被重试。**PRETRYTXNID** 的值对应第 N + 1 个被重试的请求的 TxnID。**PRETRYCRD1** 为高表示 Requester 需要等待 Completer 发回第 1 类资源的静态 P-Credit。Requester 在收到全部以上静态 P-Credit 后才可以重发对应的请求。

以图 4.2 中第 N + 2 个重试消息为例，假设该请求需要第 0 类资源与第 1 类资源。此时 Completer 内记录 Requester 持有若干个第 1 类资源的 P-Credit，但不持有任何第 0 类资源的 P-Credit，且无法立刻为该请求分配任何第 0 类资源，决定通知 Requester 该请求被重试。决定通知 Requester 该请求被重试。**PRETRYTXNID** 的值对应第 N + 2 个被重试的请求的 TxnID。**PRETRYCRD0** 为高表示 Requester 需要等待 Completer 发回第 0 类资源的静态 P-Credit。虽然请求本身需要第 1 类资源，但重试消息中 **PRETRYCRD1** 为低表示 Completer 有为该请求动态预留的空闲的第 1 类资源，且能够立刻静态地分配给该请求，那么此时 Requester 不再需要等待 Completer 发回第 1 类资源的静态 P-Credit。

14.3 基于冲突的请求 Retry

14.4 参考设计

14.4.1 通道

以 TXREQ 通道为例，本规范提供三种类型的通道的参考设计。对于 RXSNP 来说，也可以参考这三种类型的设计，仅翻转各个通道信号的方向即可。

需要注意的是，每一个通道采用什么样的设计通常情况下是**由 Completer 的设计决定**的。

14.4.1.1 动态流控通道

对于支持动态 P-Credit 与 Retry 机制的端口，其设计如图 4. 所示。

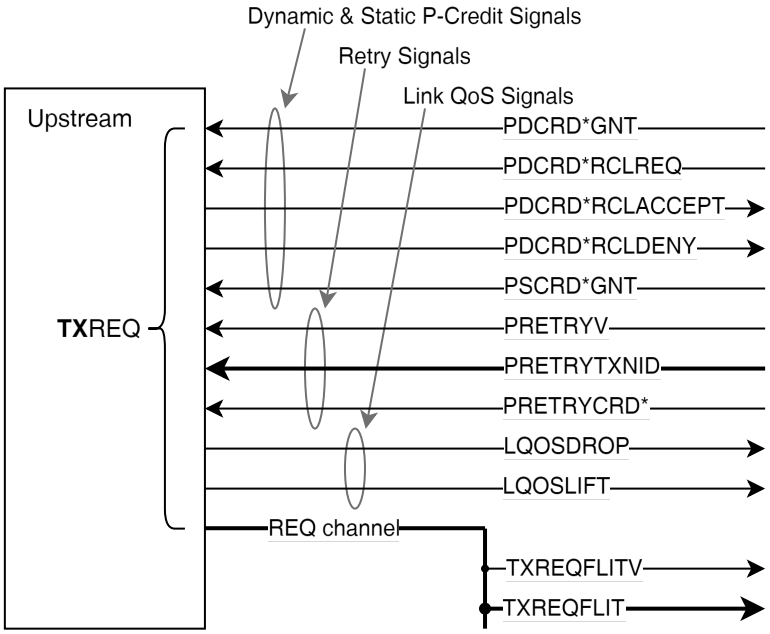


图 14.6: 动态流控通道参考设计

如图 4. 的动态流控通道能够最大程度地让 Completer 灵活的在多个 Requester 之间分配资源，并且根据各个 Requester 报告的链路 QoS 等信息动态地、有倾向地将资源分配给各个 Requester，以达到动态平衡，或为某个 Requester 提供响应时间的大概保证。关于链路 QoS 请见后文对应小节。

但此种设计会提高 Requester 与 Completer 内部的流控逻辑复杂度，并要求 Requester 与 Completer 都支持 Retry 机制。

14.4.1.2 静态流控通道

对于仅使用静态 P-Credit 进行简单单向流控的端口，其设计如图 4. 所示。

如图 4. 的静态流控通道可以仅使用静态 P-Credit 在 Requester 与 Completer 之间实现最简单的流控。通常来说，建议此时静态 P-Credit 直接与 Completer 内部的消息接收 Buffer 相关, 此种设计可以被视为 Ready-Valid 握手的一种简单替代。此时不建议再将静态 P-Credit 与 Completer 中的具体协议资源绑定，可能会导致多个空闲的 Requester 提前占用了 Completer 内的资源而无法释放，导致资源的浪费。

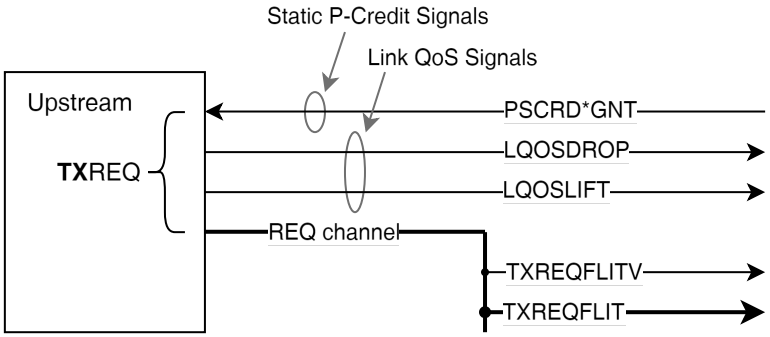


图 14.7: 静态流控通道参考设计

此种设计的端口不支持亦不需要支持 Retry 机制。

14.4.1.3 无阻塞通道

对于不进行流控的端口，其设计如图 4. 所示。

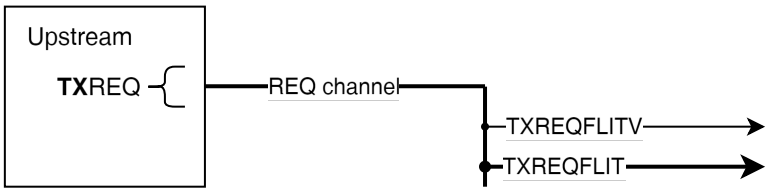


图 14.8: 无阻塞通道参考设计

此种设计在端口上不会产生任何因流控而造成的阻塞。此种设计的端口需要保证在任何情况下都能接收消息，并且总能有足够的资源完全完成事务，可能会需要 Completer 内部预留较多的资源，或完全流水化地处理。

14.4.2 端口

以 Type 1 上游端口为例，本标准提供如图 4. 的参考设计。

图 4. 所示的参考设计在 TXREQ 通道上使用了完全的动态流控通道，支持动态 P-Credit、重试机制以及链路 QoS 反馈。而 RXSNP 通道上使用了静态流控通道，仅使用静态 P-Credit 进行简单的流控。

在一个 L1-L2 缓存系统中，对于 TXREQ 通道来说，通常有多个 Requester 而只有单个 Completer。那么对于下游的 Completer 来说，在 RXREQ 通道中接收的请求是多对一的，即有多个上游 Requester 可以同时发起请求。此时下游 Completer 就需要合理地将资源分配给各个不同上游 Requester，以保证负载平衡与响应时间，所以此处使用了完整的动态流控通道。在下游 Completer 内部，应该根据需求实现 Credit 的分发算法，并处理来自各个上游 Requester 的动态反馈。

同时，在一个 L1-L2 缓存系统中，对于 RXSNP 通道来说，通常只有单个 Requester 而有多个 Completer。这是由于 Snoop 请求是从下游发送给上游的，并且 Snoop 的动作通常是一对多的。对于必须要向上游发送的 Snoop 来说，其请求的处理资源集中在各个上游 Completer 内部。那么对于每一个上游 Completer 的 Snoop 请求都是一对一的，不存在形如 TXREQ 通道中下游 Completer 的多对一请求处理情况，所以不需要复杂的反馈与流控算法来保证负载平衡与响应时间。同时，亦不建议对于 Snoop 请求实现 Retry 机制，因为其本身在各个通道中有最高的处理优先级，可能会导致 Snoop 请求的定序流程变得不必要地复杂。

此种设计的基本原则在于，实现复杂的流控算法必然会导致 Completer 内部的流控逻辑变得复杂，但是对于大部分设计情况（单核独享的 L2 或者是多核 Cluster 下的共享 L2）来说，会存在多个上游节点，但总只会存

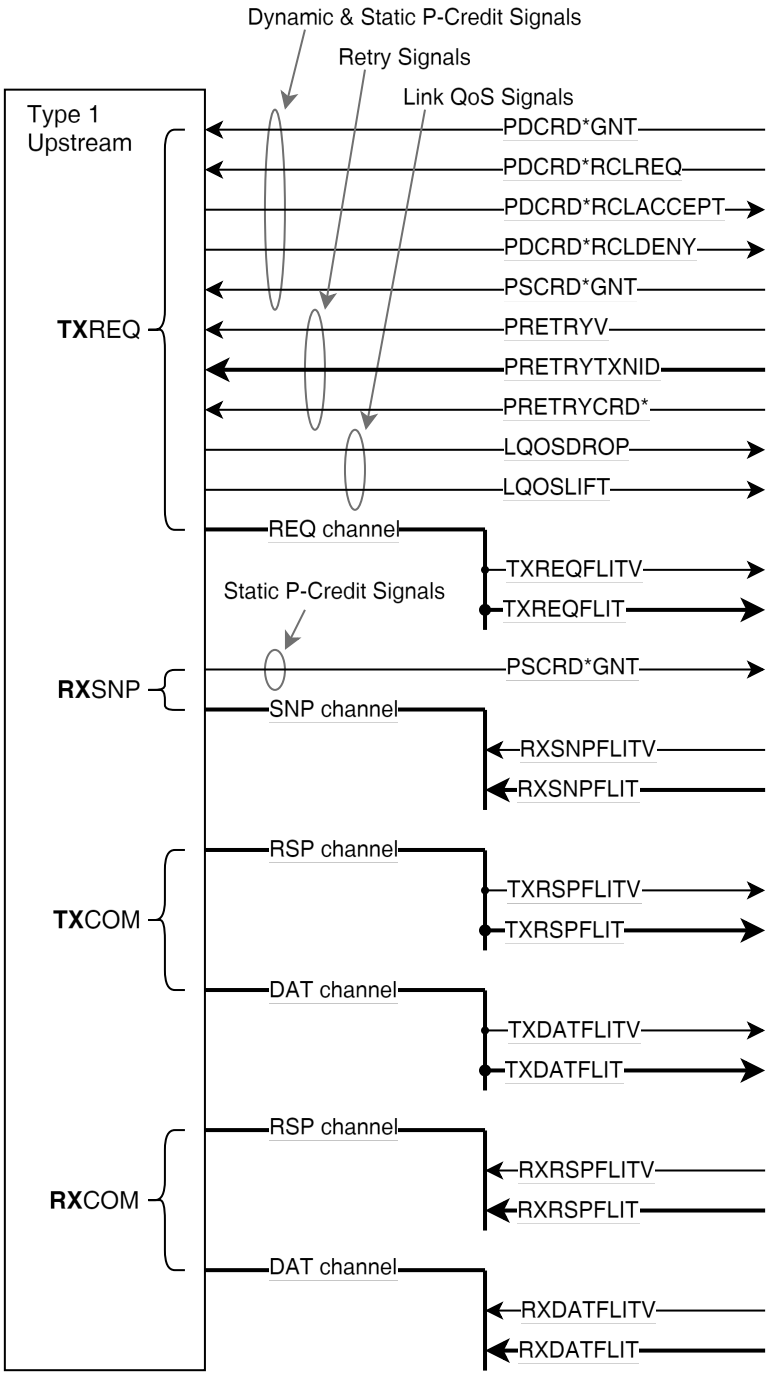


图 14.9: 上游端口参考设计

在一个下游节点。那么设计上应该会倾向于使得数量较多的上游节点的实现尽可能简单，而同时为了保证设计简洁、性能等，在数量总是很少的节点或更能够获得全局信息的下游节点上实现一些较为复杂的特性。

14.4.3 P-Credit 资源绑定

本标准提供的 Completer 侧 P-Credit 资源绑定参考设计如图 4. 所示。

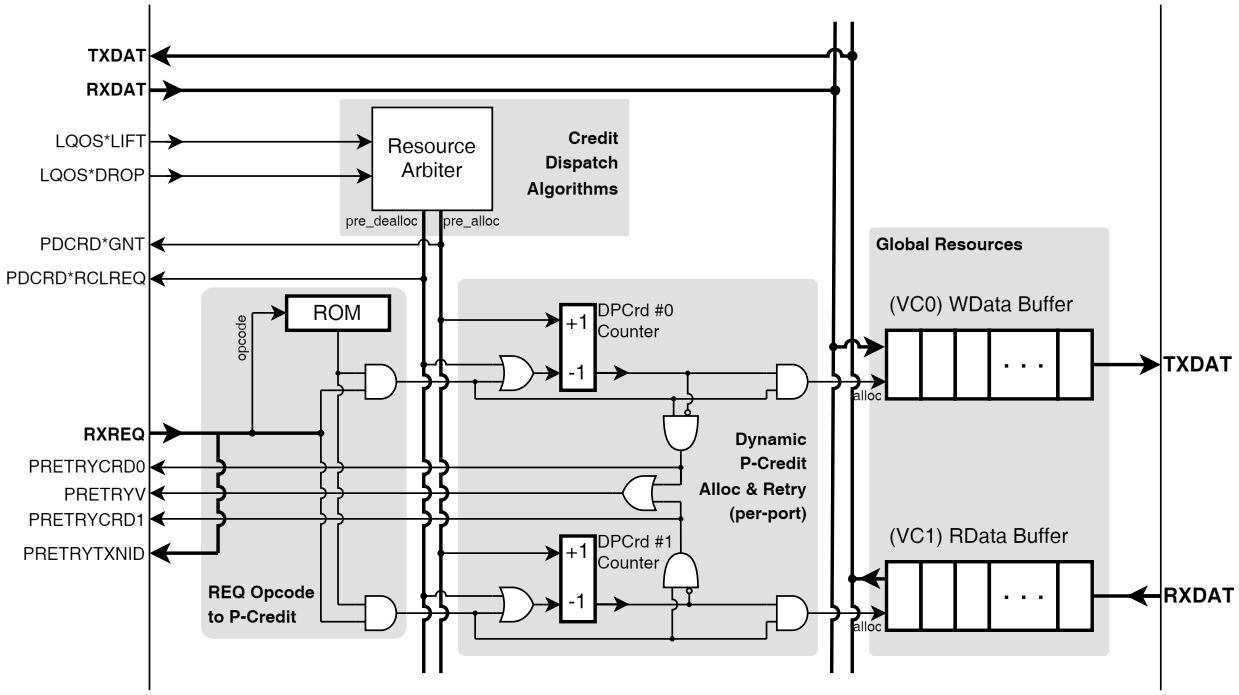


图 14.10: Completer 侧 P-Credit 资源绑定参考设计

假设对于 REQ 通道，Completer 内有两种资源：

- 写缓冲区（图中名称为 `WData Buffer`）
- 读缓冲区（图中名称为 `RData Buffer`）

其中写缓冲区用来暂存来自上游 RXDAT 的数据，并用来准备向更下游 TXDAT 转发。其中读缓冲区用来暂存来自更下游 RXDAT 的数据，并用来准备向上游 TXDAT 转发。不同的请求可能需要不同的资源。例如写回请求可能需要写缓冲区，而不需要读缓冲区。那么对于该设计，我们在上游与下游之间可以定义一种共识，将各种不同的资源映射到 P-Credit 上，如表 4.。而更下游的组件不参与本标准中的 P-Credit 交互。

表 14.4: Completer 侧 P-Credit 资源映射表参考设计

P-Credit	信号组	资源	说明
#0	**P*CRD0***	WData Buffer	写缓冲区
#1	**P*CRD1***	RData Buffer	读缓冲区

同时上游下游之间对于各个请求需要的资源也必须定义一种共识，将各种请求映射到不同的 P-Credit 上。本标准提供的参考设计仅列举了部分请求类型，如表 4.。

表 14.5: 请求与 P-Credit 资源映射表参考设计

请求类型	P-Credit #0	P-Credit #1	说明
ReadOnce	-	Y	非持久一致性读，只需要读缓冲区资源
ReadUnique	Y	Y	持久一致性读，可能产生替换，需要读缓冲区、写缓冲区资源
WriteUniqueFull	Y	-	非持久一致性写，只需要写缓冲区资源
WriteBackFull	Y	-	缓存行写回，只需要写缓冲区资源
...			

根据表 4. 的设计，意味着 Requester 在发送 ReadOnce 之前，自己必须至少持有 1 个 #0 P-Credit；在发送 ReadUnique 之前，自己必须至少持有 1 个 #0 P-Credit 和 1 个 #1 P-Credit。其它的请求类型以此类推。

本节提供的参考设计中的各种映射表仅作示例作用，不作为任何本标准中的硬性规定。实际设计中这些内容都由具体实现决定，仅需要符合规范中已描述的内容即可。

14.4.4 基于地址冲突的请求 Retry

15 链路 QoS

在 Compact CHI 规范中，Requester 可以通过使用链路 QoS 来动态地提示 Completer 在不同 Requester 之间分配动态、静态 P-Credit 的偏好或优先级。Completer 允许但不建议忽略来自 Requester 的链路 QoS 信号。

链路 QoS 使用独立的信号通道来传递信息，其信号名如下：

- **LQOSLIFT**
- **LQOSDROP**

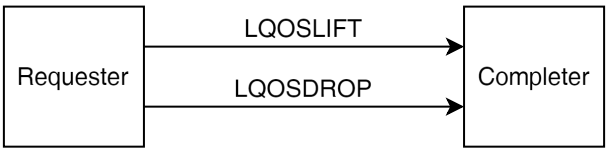


图 15.1: 链路 QoS 的传递信号

各个信号如图 4. 形式在 Requester 与 Completer 之间连接，其说明见表 4.：

表 15.1: 链路 QoS 信号列表

信号名	信号源	功能说明
LQOSLIFT	Requester	高有效，表示当前 Requester 的链路 QoS 数值增加一次
LQOSDROP	Requester	高有效，表示当前 Requester 的链路 QoS 数值减少一次

链路 QoS 的信号值允许如表 4. 的组合。

表 15.2: 链路 QoS 信号值组合

动作说明	**LQOSLIFT**	**LQOSDROP**	Completer 行为
QoS 不变	0	0	-
QoS 降低	0	1	`LQOS[Requester]--`
QoS 提高	1	0	`LQOS[Requester]++`
QoS 不变	1	1	`LQOS[Requester] = LQOS[Requester]`

每一个有效的链路 QoS 信号都只会持续一个周期，其时序图可能如图 4. 所示。

Completer 与 Requester 之间必须对链路 QoS 值的最大值达成约定，以防止出现 QoS 值的上溢出或下溢出。本规范不规定、限定 Completer 对于 QoS 值的利用方式。更大的链路 QoS 值不一定总是表示更高的链路

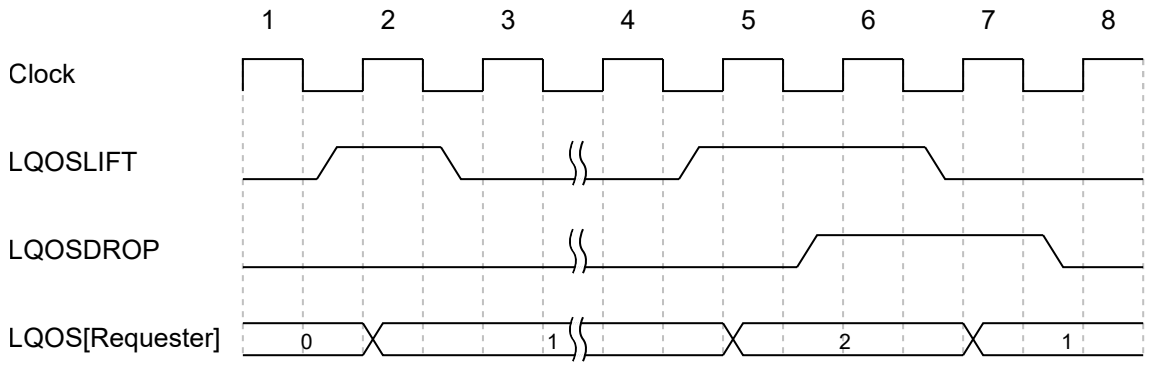


图 15.2: 链路 QoS 信号时序示例图

QoS 等级，链路 QoS 值表示的 QoS 等级可以在不同的 Requester 之间有差异，这些内容完全是由具体实现决定的。

通常来说，对于有更高链路 QoS 的 Requester，Completer 会更加倾向于将空闲的资源以动态 P-Credit 的形式分配给该 Requester，而会更少地选择其它拥有更低链路 QoS 的 Requester。使用 Weighted RR 算法可能是一种最简单的实现方式。在实现 QoS 算法时，考虑优先级差异的同时，必须保证所有的 Requester 都不会出现过长时间的资源饥饿情况。

15.1 与 P-Credit 耦合

如果需要实现更加精细的链路 QoS 管理，可以使用多个链路 QoS 传递通路，并与具体的 P-Credit 进行耦合。与 P-Credit 耦合的链路 QoS 使用独立的信号通道来传递信息，每一组信号与相应的 P-Credit 对应，其信号名如下:

- **LQOS{n}LIFT**
- **LQOS{n}DROP**

其信号列表可能如表 4. 所示。

表 15.3: 与 P-Credit 耦合的链路 QoS 信号列表示例

信号名	信号源	功能说明
PDCRD0*	*	0 号动态 P-Credit 信号组
PSCRD0*	*	0 号静态 P-Credit 信号组
LQOS0LIFT	Requester	高有效，表示当前 Requester 的 0 号 P-Credit 的链路 QoS 数值增加一次
LQOS0DROP	Requester	高有效，表示当前 Requester 的 0 号 P-Credit 的链路 QoS 数值减少一次
...		

其信号传递与值规则与上文一致。

15.2 参考设计

15.2.1 Requester 侧等待率相关的链路 QoS

一种比较简单、直观的链路 QoS 值传递方式是将其与 Requester 侧的请求等待情况相关联。

链路 QoS 值调整的本质是让 Requester 能够告知 Completer 自己有多强烈的意愿希望更早的被分配到资源。那么，通常来说，对于多个拥有相当 Outstanding 能力的 Requester，其因资源不足而产生阻塞、等待的请求数量越多，其就更有可能在某个、多个请求上因为某个资源的不足而产生更长的等待时间。所以将等待情况与链路 QoS 关联是一种比较直观的反馈方式。

本标准提供的参考设计如图 4. 所示。

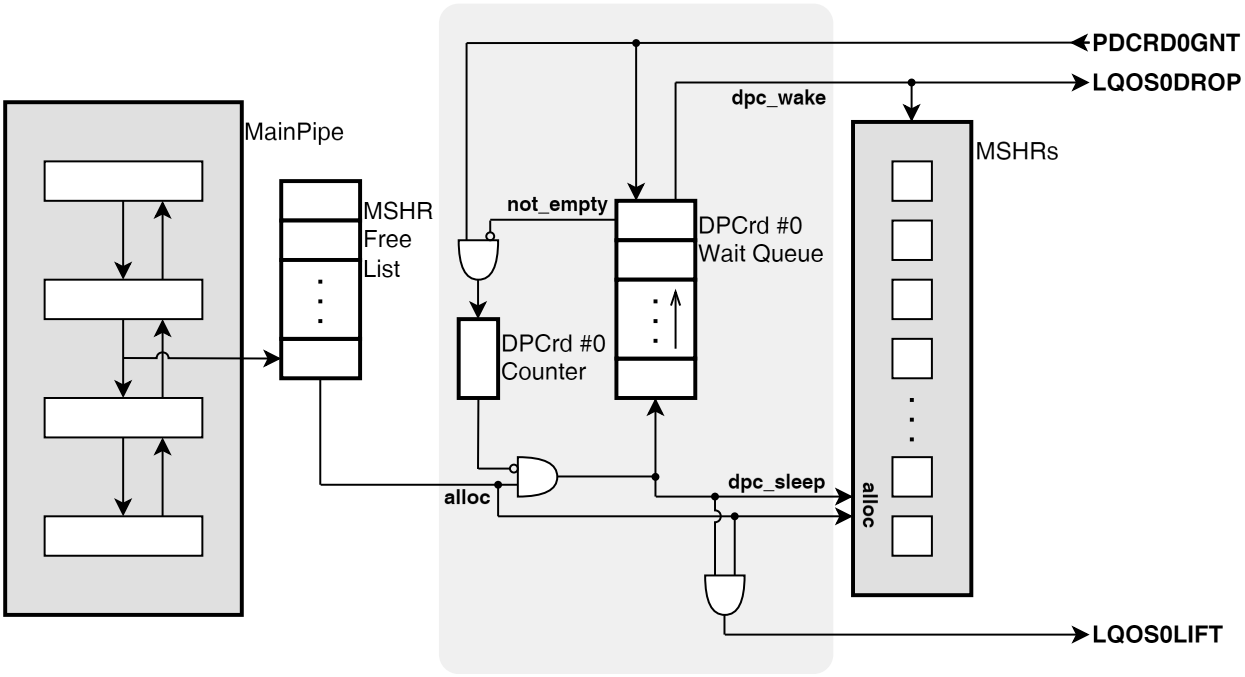


图 15.3: Requester 侧等待率相关链路 QoS 参考设计

1. 链路 QoS 值的提高

以图 4. 中的设计为例。在主流水线因收到新的任务而分配 MSHR 项时，需要先查询自己是否持有相应的动态 P-Credit。参考设计中使用一个计数器来保存目前 Requester 中持有的动态 P-Credit 数量（图中部件名称为 `DPCrd #0 Counter`），并使用一个队列来保存目前 Requester 中因动态 P-Credit 数量不足而正在等待的 MSHR 项（图中部件名称为 `DPCrd #0 Wait Queue`）。如果此时 Requester 对应的 P-Credit 计数器值为 0，那么在分配 MSHR 时就需要将其标记为暂时不可向下游发送，即进入睡眠状态（图中信号名为 `dpc_sleep`），并将向等待队列写入当前 MSHR 的指针值。在有 MSHR 因此进入睡眠状态时，同时将 **LQOS0LIFT** 信号置为有效，**提高链路 QoS 值**。

2. 链路 QoS 值的降低

以图 4. 中的设计为例。在 Requester 收到来自 Completer 的相应动态 P-Credit 时，会将其优先分配给正在等待队列中的 MSHR 项。完成分配时就会将对应的 MSHR 项唤醒（图中信号名为 `dpc_wake`）。在有 MSHR 因此被唤醒时，同时将 **LQOS0DROP** 信号置为有效，**降低链路 QoS 值**。

实际设计中也可以将动态 P-Credit（因 Requester 持有资源不足而无法发出的请求）与静态 P-Credit（因被 Completer 重试后还未收到资源而无法发出的请求）同时纳入链路 QoS 的更新逻辑中。